XPoster: Coding Instructions

PS1 ↔ Python JSON-Lines Protocol v1 and full implementation plan (version 4.0, 17 October 2026)

What this document is

A step-by-step build specification for the XPoster application: a PowerShell GUI that owns the workflow, and a Python engine that owns all communication with X. It replaces the earlier draft PDF. Follow it top to bottom. Complete, ready-to-use source is in Appendix A (Python), Appendix B (PowerShell) and Appendix C (protocol conformance test).

Words **MUST** and **MUST NOT** are binding. Items tagged **[ADAPT]** depend on parts of your original design that this document could not see; each one names the single place to change.

1. Architecture, locked decisions and assumptions 3

1.1 Architecture . . . 3
1.2 Locked values (do not rename) . . . 3
1.3 Assumptions and [ADAPT] points . . . 3
1.4 Environment . . . 4
1.5 Setup files and how to launch . . . 4

2. Build order and acceptance gates 4

2.1 Files and locations . . . 5

3. Protocol v1 specification 5

3.1 Transport rules . . . 5
3.2 Request (PS1 → Python) . . . 5
3.3 Events (Python → PS1) . . . 6
3.4 State machine (implemented in PowerShell) . . . 6
3.5 Terminal events, exit and draining . . . 7
3.6 Timeouts . . . 7
3.7 Error codes . . . 7
3.8 Recovery rules . . . 8
3.9 Golden transcripts . . . 8
3.10 Alternatives rejected and future scope . . . 11

4. Python engine instructions (XPoster.py) 11

4.1 Command line . . . 11
4.2 Processing order (implemented in run()) . . . 11
4.3 Validation limits . . . 11
4.4 Real backend: X API v2 flow . . . 12
4.5 Credentials file . . . 12
4.6 Mock backend and fault flags . . . 13

5. PowerShell instructions (XPoster.ps1) 13

5.1 Structure . . . 13
5.2 Process handling and threading (the key design decision) . . . 13
5.3 StartPost sequence . . . 14

5.4 Rules the router MUST follow . . . 14
5.5 GUI rules . . . 14
5.6 Logging . . . 14

6. Logging and security rules **14**

7. Test plan **15**

7.1 Running the conformance harness . . . 15
7.2 Test matrix . . . 15
7.3 PowerShell-side tests . . . 15
7.4 Real-API smoke test (Phase 6) . . . 15
7.5 Final acceptance checklist . . . 16

8. Decision log: what changed from the earlier draft **16**

9. Review history **16**

Review 1: fidelity to the earlier draft and layout . . . 17
Review 2: execution and failure review . . . 17
Review 3: handoff completeness and print quality . . . 17

Appendix A. XPoster.py (complete) **19**

Appendix B. XPoster.ps1 (complete) **29**

Appendix C. xposter_conformance.py (complete) **41**

1. Architecture, locked decisions and assumptions

1.1 Architecture

One PowerShell process, one Python process, three redirected standard streams. No sockets, no port per media file, no temporary transport files.

Channel	Direction	Content
stdin	PS1 → Python	Exactly one complete post request: one compact UTF-8 JSON line, then the pipe is closed.
stdout	Python → PS1	Protocol events only: one compact JSON object per line, ASCII-only.
stderr	Python → PS1	Human-readable diagnostics only. Never parsed as protocol.

PowerShell owns	Python owns
GUI, application state, workflow, process lifecycle, protocol parsing and validation, progress display, timeouts, final result handling, logging, post-record export.	Request validation, credentials and X API layer, text preparation, sequential media upload, progress emission, final post creation, protocol event emission.

1.2 Locked values (do not rename)

Group	Values
Events	`READY` `TEXT_COMPLETE` `PROGRESS` `MEDIA_COMPLETE` `POSTING` `SUCCESS` `ERROR`
Request fields	`ProtocolVersion` `RequestId` `Text` `TotalFiles` `Media[]` with `MediaId` `FileIndex` `FileName` `FilePath`. There is **no** `FileName1`, `FileName2`, ...
Config keys (X_Config.txt)	`PythonPath` `PythonScriptPath` `DataRootPath` `OperationalPath` `TextFilesPath` `MediaFilesPath` `ExcelFilePath` `KeysFilePath` `DebugLogReadMePath` `DebugLogFunctionPath` `DebugLogMode` `DebugLogOutputPath`. The value `Per run` stays as is.
GUI state names	`CurrentFileIndex` `TotalFiles` `CurrentFileProgress` `CurrentFileName` `PublishedPostId` `PublishedPostUrl`
PowerShell functions	`MainController` `ReadConfig` `DetermineStartupMode` `InitializeGui` `AttachMedia` `RemoveMedia` `UpdatePostButtonState` `UpdateGui` `UpdateUploadProgress` `StartPost` `ExportPostRecord` `RunPythonEngine` `HandlePythonResponse`
Behavior	Up to 4 media items. Sequential upload. Selected media are local paths only until Post is pressed. Text may be empty, media may be empty, never both. Credentials live in X_Keys.txt and are never part of the protocol. No automatic repost retries.

> **Config file path**
>
> The config file is at `Z:\InsPro\Tools\Data\X_Post\Operational\X_Config.txt`. The line shown as `2DebugLogReadMePath` in the earlier PDF was a footnote number fused to the key by the PDF renderer; the real key is `DebugLogReadMePath`. The PowerShell script holds the config path as its only hard-coded path (`$script:ConfigPath`) and passes it to Python with `--config`.

1.3 Assumptions and [ADAPT] points

The original design record that the earlier draft cites was not available when this document was written. Where it mattered, a working default was chosen. Each default is isolated:

Topic	Default used here	Change it in
Debug log call	`Write-DebugLog -Message` is called inside a try/catch. Use the real function and signature from DebugLogReadMe.txt.	`WriteDebugOutput` (PS1)
X_Keys.txt format	NAME=VALUE lines: `API_KEY`, `API_SECRET`, `ACCESS_TOKEN`, `ACCESS_TOKEN_SECRET` (OAuth 1.0a user keys; they do not expire).	`load_keys`, `XBackend.__init__` (Python)
Post record / Excel	`ExportPostRecord` writes `<RequestId>.txt` to TextFilesPath, copies media into MediaFilesPath, and appends status rows to `ExcelFilePath\PostHistory.csv` (opens in Excel).	`ExportPostRecord`, `WriteHistoryRow` (PS1)

Topic	Default used here	Change it in
Startup checks	`DetermineStartupMode` only checks that Python, XPoster.py and the keys file exist.	`DetermineStartupMode` (PS1)
GUI layout	Minimal WinForms form wired to the locked function names.	`InitializeGui` (PS1)
Account tier	Text limit 280 characters; video cap 512 MB.	`MAX_TEXT_CHARS`, `MAX_VIDEO_BYTES` (Python)
Timeouts	Startup 30 s, idle 90 s, total 30 min, drain grace 5 s.	constants at top of PS1

1.4 Environment

- Windows, Windows PowerShell 5.1 or PowerShell 7 (WinForms). All PowerShell code avoids syntax newer than 5.1 (no `?.`, ternary, `&&`), and the embedded C# is C# 5.
- Python 3.13 at the configured `PythonPath`. Mock mode needs only the standard library. The real engine additionally needs `pip install requests requests-oauthlib`.
- The Python process MUST be started directly (`UseShellExecute=false`), never through `cmd.exe`, with `-u` (unbuffered) as a defensive extra to the explicit flushing.

1.5 Setup files and how to launch

X_Config.txt (existing, unchanged; one `Key=Value` per line, split on the first `=`):

```
PythonPath=C:\Python\Python313\python.exe
PythonScriptPath=Z:\InsPro\Tools\Py\XPoster.py
DataRootPath=Z:\InsPro\Tools\Data\X_Post
OperationalPath=Z:\InsPro\Tools\Data\X_Post\Operational
TextFilesPath=Z:\InsPro\Tools\Data\X_Post\TextFiles
MediaFilesPath=Z:\InsPro\Tools\Data\X_Post\MediaFiles
ExcelFilePath=Z:\InsPro\Tools\Data\X_Post\ExcelFile
KeysFilePath=Z:\InsPro\Tools\Data\X_Post\Operational\X_Keys.txt
DebugLogReadMePath=Z:\InsPro\Tools\Ps1\ReadMe\DebugLogReadMe.txt
DebugLogFunctionPath=Z:\InsPro\Tools\Ps1\DebugLog.ps1
DebugLogMode=Per run
DebugLogOutputPath=Z:\InsPro\Tools\Logs\XPoster\xxxxxxx.log
```

The `xxxxxxx` in the log path is a placeholder. Let DebugLog.ps1 substitute the per-run name as its ReadMe describes; XPoster.ps1 passes the value through untouched.

X_Keys.txt (new in this version; placeholders shown, real values never go into a document or log):

```
API_KEY=<consumer key>
API_SECRET=<consumer secret>
ACCESS_TOKEN=<user access token>
ACCESS_TOKEN_SECRET=<user access token secret>
```

The X app must have **Read and write** permission, and the access token must be generated after that setting is applied, or X answers HTTP 403.

```
REM one-time install of the real-engine packages
C:\Python\Python313\python.exe -m pip install requests requests-oauthlib

REM launch the application (WinForms needs a single-threaded apartment)
powershell.exe -STA -ExecutionPolicy Bypass -File Z:\InsPro\Tools\Ps1\XPoster.ps1
```

Developer switch: set `$script:ExtraPythonArgs = '--fake'` (or `'--fake --fault upload_fail_2'`) at the top of XPoster.ps1. It MUST be an empty string in production.

2. Build order and acceptance gates

Build in layers so the protocol is proven before any real X call is made. Do not start a phase until the previous gate passes.

#	Phase	Do	Gate (must pass)
1	Constants	Copy event names, codes, field names from section 3 into both files.	Same spelling in PS1 and Python.
2	Python mock engine	Install Appendix A. Run Appendix C.	Conformance harness: all tests PASS.

#	Phase	Do	Gate (must pass)
3	PS1 transport	Appendix B: runner, pump timer, router, with `$script:ExtraPythonArgs = '--fake'`.	Fake post completes; GUI never freezes.
4	PS1 fault tests	Repeat tests D to O of section 7 through the GUI using `--fault`.	Every test ends in the expected code and a usable GUI.
5	Logging	Wire `WriteDebugOutput` to DebugLog.ps1.	One RequestId can be traced start to finish; no secrets in the log.
6	Real upload	Create X_Keys.txt, install packages, run the smoke test (section 7.4).	One text-only post, then one image post, succeed.
7	Integration	Full GUI workflow with 1 to 4 images, error recovery, window close.	Section 7.5 checklist complete.

2.1 Files and locations

File	Location	Role
XPoster.py	`Z:\InsPro\Tools\Py\XPoster.py`	Python engine (Appendix A)
XPoster.ps1	`Z:\InsPro\Tools\Ps1\XPoster.ps1`	PowerShell controller and GUI (Appendix B)
xposter_conformance.py	next to XPoster.py	Protocol test client (Appendix C)
X_Config.txt, X_Keys.txt	`...\X_Post\Operational\`	Configuration and credentials

3. Protocol v1 specification

3.1 Transport rules

- **Framing.** One JSON object per physical line (JSON Lines). Compact JSON only: no pretty printing and no raw newline inside a record. Newlines inside `Text` travel as `\n` escapes.
- **Encoding.** Request: UTF-8 without BOM, written to the pipe as bytes. Events: Python emits ASCII-only JSON (`ensure_ascii=True`), so non-ASCII characters appear as `\uXXXX` escapes and no stdout code-page setting can corrupt them. Line terminator is a bare LF; PowerShell tolerates CRLF.
- **One request per process.** PS1 starts Python, writes one line, flushes, closes stdin. Python handles that request and exits. Cancellation, queues and a persistent daemon are out of scope for v1.
- **stdout is protocol only.** A stray `print()` is a protocol violation. Diagnostics go to stderr.
- **Flush.** Python flushes after every event. The only writer of protocol lines is `emit_event`.
- **Sequential.** Media are uploaded one after another in `FileIndex` order. Maximum 4 media.

3.2 Request (PS1 → Python)

Field	Type	Rule
ProtocolVersion	integer	Must equal 1.
RequestId	string	Non-empty, unique per posting attempt. PS1 uses `yyyyMMdd_HHmmss_fff`. Opaque to Python.
Text	string	May be empty or whitespace. Sent exactly as typed.
TotalFiles	integer	0 to 4.
Media	array	Length must equal `TotalFiles`; `[]` when 0.
Media[].MediaId	integer	Unique local ID, 1 to 4. Not the same thing as `XMediaId`.
Media[].FileIndex	integer	Sequential position 1 to `TotalFiles`; equals array position.
Media[].FileName	string	Display name only.
Media[].FilePath	string	Absolute path readable by Python (the exported copy in MediaFilesPath).

`MediaId` is local and fixed before Python starts. `XMediaId` is the identifier returned by X and is known only at `MEDIA_COMPLETE`. PS1 keeps the map `MediaIds[MediaId] = XMediaId`. All IDs from X (`XMediaId`, `PostId`) are strings and MUST NOT be converted to numbers.

The whole post is in the request. A `PROGRESS` event describes only the one media item whose state changed; the other items were already sent in the request.

3.3 Events (Python → PS1)

Every event contains `Event` and `RequestId`. `RequestId` is `null` only in an `ERROR` for a request so broken that its ID could not be read; PS1 MUST accept exactly that case.

Event	Additional required fields	Optional	Meaning
READY	ProtocolVersion	-	Request fully accepted: JSON parsed, version accepted, schema and files validated, credentials loaded. After READY only runtime or remote failures can occur.
TEXT_COMPLETE	TextPresent (bool), CharacterCount	-	Text accepted for the pending post. Always sent, even for media-only. NOT published yet.
PROGRESS	MediaId, FileIndex, TotalFiles, Percent (0-100), FileName	BytesUploaded, BytesTotal	State of one media upload. The same Percent may repeat as a heartbeat.
MEDIA_COMPLETE	MediaId, FileIndex, TotalFiles, FileName, XMediaId	-	One upload finished.
POSTING	MediaCount	-	All uploads done; final post request is about to be sent.
SUCCESS	PostId, PostUrl	-	Post exists. Terminal.
ERROR	Code, Stage, Message, Retryable (bool)	MediaId, FileIndex, HttpStatus	Failure. Terminal. Message never contains secrets.

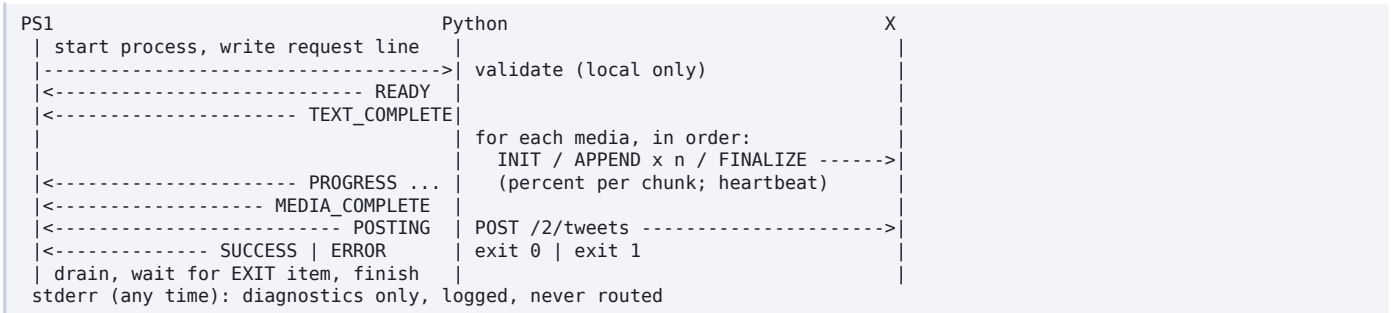
3.4 State machine (implemented in PowerShell)

PS1 tracks a phase and checks every event against this table. Any other combination is `PROTOCOL_SEQUENCE_ERROR`. `Next` is the 0-based index of the media item currently being uploaded.

Event	Allowed in phase	Extra checks	New phase
READY	Starting	-	Ready
TEXT_COMPLETE	Ready	-	Media
PROGRESS	Media, Next < TotalFiles	MediaId == Media[Next].MediaId	Media
MEDIA_COMPLETE	Media, Next < TotalFiles	same MediaId check; XMediaId non-empty; then Next += 1	Media
POSTING	Media	Next == TotalFiles and MediaCount == TotalFiles	Posting
SUCCESS	Posting	PostId and PostUrl present	Draining
ERROR	Starting, Ready, Media, Posting	-	Draining

A `PROGRESS` before `MEDIA_COMPLETE` is not required (a fast upload may skip straight to `MEDIA_COMPLETE`), but the mock and real engines always send `Percent` 0 first and 100 last.

Normal sequence



3.5 Terminal events, exit and draining

- Exactly one terminal event (`SUCCESS` or `ERROR`) per run; Python sends nothing after it (`emit_event` suppresses later events). Python exits 0 after SUCCESS and 1 after ERROR.
- On a terminal event PS1 enters phase **Draining**: later stdout lines are logged and ignored, the GUI shows "Finishing...", and the operation is finalized when the process-exit item arrives or after the 5 s grace period (then the process is killed). Only then is the Post button re-enabled.
- The C# runner posts its `EXIT` item to the queue only after `WaitForExit()` returns, which in .NET happens after stdout and stderr reach end-of-file. Therefore every stdout line is always processed **before** the exit item. A process exit with no terminal event is `PYTHON_EXITED_UNEXPECTEDLY`.

3.6 Timeouts

Name	Default	Triggers `OPERATION_TIMEOUT` when
StartupTimeoutSec	30	no READY within 30 s of process start
IdleTimeoutSec	90	no stdout line for 90 s in phases Ready, Media or Posting. The real engine sends a heartbeat PROGRESS while waiting for X to process media.
TotalTimeoutSec	1800	the whole operation exceeds 30 minutes
DrainGraceSec	5	Python still running 5 s after a terminal event (killed, not an error)

A timeout stops the operation, kills Python and records `OPERATION_TIMEOUT`. If the phase was Posting the outcome is treated as unknown (section 3.8).

3.7 Error codes

Python-owned (sent in an `ERROR` event). Stage is one of RequestValidation, Startup, Auth, MediaUpload, PostCreate, Internal.

Code	Stage	Retryable	Meaning / recovery
MALFORMED_REQUEST_JSON	RequestValidation	No	stdin was empty, not UTF-8, not JSON, or not an object. RequestId is null.
UNSUPPORTED_PROTOCOL_VERSION	RequestValidation	No	ProtocolVersion is not 1.
INVALID_REQUEST	RequestValidation	No	Wrong types, missing fields, duplicate MediaId, non-sequential FileIndex, relative path.
EMPTY_POST	RequestValidation	No	Blank text and no media.
MEDIA_COUNT_INVALID	RequestValidation	No	TotalFiles outside 0-4 or not equal to Media length.
MEDIA_FILE_NOT_FOUND	RequestValidation	No	File missing. User re-selects it.
MEDIA_FILE_UNREADABLE	RequestValidation / MediaUpload	No	Cannot read, or zero bytes.
MEDIA_TYPE_UNSUPPORTED	RequestValidation	No	Extension not in the supported list.
MEDIA_TOO_LARGE	RequestValidation	No	Over the size cap for its type.
MEDIA_COMBINATION_INVALID	RequestValidation	No	A GIF or video together with any other media.
TEXT_TOO_LONG	RequestValidation	No	More than `MAX_TEXT_CHARS`.
CONFIG_ERROR	Startup	No	Config unreadable, KeysFilePath missing, or Python packages missing.
CREDENTIAL_ERROR	Startup	No	X_Keys.txt unreadable or a required name is missing.
AUTH_FAILED	Auth / MediaUpload / PostCreate	No	HTTP 401 from X.
RATE_LIMITED	MediaUpload / PostCreate	Yes	HTTP 429. Retry later, manually.
NETWORK_ERROR	MediaUpload / PostCreate	Yes	Connection failed before the request was sent.
MEDIA_UPLOAD_FAILED	MediaUpload	5xx only	Any other upload or processing failure.

Code	Stage	Retryable	Meaning / recovery
POST_CREATE_FAILED	PostCreate	No	X refused the post (4xx). Nothing was created.
POST_RESULT_UNKNOWN	PostCreate	No	The post request may have reached X. Never auto-retry. Check X manually.
INTERNAL_ERROR	Internal	No	Unexpected exception. Details go to stderr only.

PowerShell-owned (generated locally when Python cannot say it itself).

Code	Raised when
PYTHON_START_FAILED	The process could not be started (bad path, missing runtime).
PYTHON_EXITED_UNEXPECTEDLY	Exit item arrives with no terminal event. Message carries the exit code and the last stderr lines.
PROTOCOL_INVALID_STDOUT_JSON	A non-blank stdout line is not valid JSON.
PROTOCOL_REQUEST_ID_MISMATCH	An event carries another RequestId (null allowed only for ERROR).
PROTOCOL_UNEXPECTED_EVENT	Unknown event, missing `Event`, or a required field is missing.
PROTOCOL_SEQUENCE_ERROR	A valid event arrives in the wrong phase or for the wrong MediaId.
OPERATION_TIMEOUT	Any timeout in section 3.6.

3.8 Recovery rules

- **No automatic repost retries, ever.** `Retryable=true` means a manual retry may make sense; PS1 never repeats an operation by itself.
- **Partial media upload.** If media 1 completed and media 2 failed: do not post, discard the operation, keep the GUI inputs. The next Post gets a new RequestId and uploads everything again.
- **Unknown outcome.** `POST_RESULT_UNKNOWN`, or any synthetic failure while the phase is Posting, sets `UnknownOutcomePending`. The error dialog says to check X first, and the next Post attempt requires an explicit confirmation.
- **Broken protocol.** After any PS1-detected protocol failure the stream is not trusted: Python is killed and the operation ends. PS1 never continues applying later lines.

3.9 Golden transcripts

These are real outputs captured from `XPoster.py --fake`. Each numbered row is **one physical line**; long lines are only wrapped for printing. They are the reference for Python output and the PowerShell parser tests.

Request, two files (single line on the pipe)

```
1    {"ProtocolVersion":1,"RequestId":"20261017_090153_412","Text":"ddddddddddd","TotalFiles":2,"Media":[{"MediaId":1,"FileIndex"
     :1,"FileName":"image1.jpg","FilePath":"Z:\\InsPro\\Tools\\Data\\X_Post\\MediaFiles\\image1.jpg"},{"MediaId":2,"FileIndex":2,
     "FileName":"image2.jpg","FilePath":"Z:\\InsPro\\Tools\\Data\\X_Post\\MediaFiles\\image2.jpg"}]}
```

Same request, pretty-printed for reading only

```
{
  "ProtocolVersion": 1,
  "RequestId": "20261017_090153_412",
  "Text": "ddddddddddd",
  "TotalFiles": 2,
  "Media": [
    {
      "MediaId": 1,
      "FileIndex": 1,
      "FileName": "image1.jpg",
      "FilePath": "Z:\\InsPro\\Tools\\Data\\X_Post\\MediaFiles\\image1.jpg"
    },
    {
      "MediaId": 2,
      "FileIndex": 2,
      "FileName": "image2.jpg",
      "FilePath": "Z:\\InsPro\\Tools\\Data\\X_Post\\MediaFiles\\image2.jpg"
    }
  ]
}
```

Two-file post: Python stdout

```
{"Event":"READY","RequestId":"20261017_090153_412","ProtocolVersion":1}
{"Event":"TEXT_COMPLETE","RequestId":"20261017_090153_412","TextPresent":true,"CharacterCount":11}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":0,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":25,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":37,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":50,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":75,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":100,"FileName":"image1.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_090153_412","MediaId":1,"FileIndex":1,"TotalFiles":2,"FileName":"image1.jpg","XMediaId":"101000000000000000"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":0,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":25,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":37,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":50,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":75,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":100,"FileName":"image2.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_090153_412","MediaId":2,"FileIndex":2,"TotalFiles":2,"FileName":"image2.jpg","XMediaId":"102000000000000000"}
{"Event":"POSTING","RequestId":"20261017_090153_412","MediaCount":2}
{"Event":"SUCCESS","RequestId":"20261017_090153_412","PostId":"9990000000000000001","PostUrl":"https://x.com/i/web/status/9990000000000000001"}
```

Text-only post: Python stdout

```
{"Event":"READY","RequestId":"20261017_091500_077","ProtocolVersion":1}
{"Event":"TEXT_COMPLETE","RequestId":"20261017_091500_077","TextPresent":true,"CharacterCount":9}
{"Event":"POSTING","RequestId":"20261017_091500_077","MediaCount":0}
{"Event":"SUCCESS","RequestId":"20261017_091500_077","PostId":"9990000000000000001","PostUrl":"https://x.com/i/web/status/9990000000000000001"}
```

Upload failure on media 2 (fault upload_fail_2): Python stdout

```
{"Event":"READY","RequestId":"20261017_091620_305","ProtocolVersion":1}
{"Event":"TEXT_COMPLETE","RequestId":"20261017_091620_305","TextPresent":true,"CharacterCount":11}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":0,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":25,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":37,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":50,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":75,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":100,"FileName":"image1.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_091620_305","MediaId":1,"FileIndex":1,"TotalFiles":2,"FileName":"image1.jpg","XMediaId":"101000000000000000"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":0,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":25,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":37,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":50,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091620_305","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":61,"FileName":"image2.jpg"}
{"Event":"ERROR","RequestId":"20261017_091620_305","Code":"MEDIA_UPLOAD_FAILED","Stage":"MediaUpload","Message":"Media upload failed.","Retryable":true,"MediaId":2,"FileIndex":2}
```

Media-only post (Text empty): Python stdout. TEXT_COMPLETE is still sent

```
{"Event":"READY","RequestId":"20261017_091600_118","ProtocolVersion":1}
{"Event":"TEXT_COMPLETE","RequestId":"20261017_091600_118","TextPresent":false,"CharacterCount":0}
{"Event":"PROGRESS","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"Percent":0,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"Percent":25,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"Percent":37,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"Percent":50,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"Percent":75,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"Percent":100,"FileName":"image1.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_091600_118","MediaId":1,"FileIndex":1,"TotalFiles":1,"FileName":"image1.jpg","XMediaId":"101000000000000000"}
{"Event":"POSTING","RequestId":"20261017_091600_118","MediaCount":1}
{"Event":"SUCCESS","RequestId":"20261017_091600_118","PostId":"9990000000000000001","PostUrl":"https://x.com/i/web/status/9990000000000000001"}
```

Empty post reaching Python (PS1 normally prevents it): defensive EMPTY_POST

```
{"Event":"ERROR","RequestId":"20261017_091700_042","Code":"EMPTY_POST","Stage":"RequestValidation","Message":"Post must contain text or at least one media item.","Retryable":false}
```

3.10 Alternatives rejected and future scope

Design	Verdict	Reason
JSON Lines over stdin/stdout	**Use**	Structured, escaped, one ordered stream, native parsers on both sides, easy to version and to read in a log.
`EVENT|Key=Value` **text**	Reject	Needs hand-made escaping for `|`, `=`, newlines and Unicode; arrays and versioning are awkward.
One socket or port per media file	Reject	Port allocation, firewall, ordering, cleanup; pipes already exist.
One socket for everything	Not for v1	Adds networking without solving a present requirement.
Parallel media upload	Not in v1	Would still fit one stdout stream (events carry MediaId), but GUI and state semantics change; treat as a separate protocol version.
Persistent Python daemon, cancel command	Not in v1	Needs command multiplexing; version it separately.

4. Python engine instructions (XPoster.py)

4.1 Command line

Argument	Purpose
`--config <path>`	Path to X_Config.txt, supplied by PS1. Python reads only `KeysFilePath` from it.
`--fake`	Dev only. Use the mock backend: no network, no credentials.
`--fault <name>`	Dev only, with `--fake`. Inject a failure (table in 4.6).

4.2 Processing order (implemented in `run()`)

1. Read one line from `sys.stdin.buffer`, decode as UTF-8 (a BOM is tolerated), parse JSON.
2. Extract `RequestId` as soon as possible so every later ERROR can echo it.
3. Validate the request (`validate_request`): version, fields, media list, then each file (exists, readable, non-empty, supported type, size cap), then EMPTY_POST, then the GIF/video rule.
4. Prepare text: convert CRLF/CR to LF only; reject over-length. Never trim or alter user text.
5. Create the backend (loads credentials in real mode).
6. Emit `READY`, then `TEXT_COMPLETE`.
7. For each media item in order: upload with a `ProgressReporter`, then emit `MEDIA_COMPLETE`.
8. Emit `POSTING`, create the post, emit `SUCCESS` and return 0.
9. Any `KnownProtocolError` becomes one `ERROR` event and return 1. Any other exception becomes `INTERNAL_ERROR`. A `BrokenPipeError` (PS1 gone) exits quietly.

> **Hard rules for Python**
>
> Never call `print()` without `file=sys.stderr`. Use `debug()` for diagnostics.
>
> Never put credentials, tokens, headers, request bodies or raw response bodies into `Message`, stderr or exceptions. X error text is limited to the `detail/title` field, trimmed to 160 characters.
>
> Only `emit_event` writes protocol lines. It flushes every time and refuses output after a terminal event.

4.3 Validation limits

Type	Extensions	X category	Cap
Image	.jpg .jpeg .png .webp	tweet_image	5 MB
Animated GIF	.gif	tweet_gif	15 MB (only item in the post)
Video	.mp4 .mov	tweet_video	512 MB application cap (only item in the post)

A post carries up to 4 images, or exactly one GIF, or exactly one video. X remains the final authority: a post can still be refused when the media is attached, which surfaces as `POST_CREATE_FAILED` with the X detail text. The text limit is the character count of the normalized text against `MAX_TEXT_CHARS` (280); X counts some characters differently, so treat this as a pre-check, not as the authority.

4.4 Real backend: X API v2 flow

Verified against the X developer documentation on 17 October 2026 (chunked media upload and create-post quickstarts). Re-check docs.x.com if X changes it. The media endpoints are not part of the post endpoint, and the older `command=INIT/APPEND/FINALIZE` form on `POST /2/media/upload` is the previous protocol: do not use it.

Step	Call	Notes
INIT	`POST /2/media/upload/initialize`, JSON body `{media_type, total_bytes, media_category}`	Response `data.id` is the `XMediaId` (string).
APPEND	`POST /2/media/upload/{id}/append`, multipart `segment_index` + `media`	2 MiB per chunk (limit 5 MB). One PROGRESS per chunk, capped at 99 until finalize.
FINALIZE	`POST /2/media/upload/{id}/finalize`	If `processing_info` is returned, poll.
STATUS	`GET /2/media/upload?command=STATUS&media_id={id}`	Wait `check_after_secs` (1 to 15 s) until `succeeded`; `failed` is MEDIA_UPLOAD_FAILED. Heartbeat PROGRESS on each poll; give up after 600 s.
POST	`POST /2/tweets`, JSON `{text, media:{media_ids:[...]}}`	Omit `text` when blank and `media` when there is no media. `data.id` is the PostId; PostUrl is `https://x.com/i/web/status/<id>`.

Authentication. OAuth 1.0a user context (four values in X_Keys.txt) through `requests_oauthlib.OAuth1Session`. It does not expire, which suits an unattended desktop tool. If you prefer OAuth 2.0 user tokens, only `XBackend.__init__` changes, but tokens expire and then need refresh logic. Timeouts are (10 s connect, 60 s read) on every call.

Situation	Code	Retryable	Notes
HTTP 429	RATE_LIMITED	Yes	Either stage.
HTTP 401	AUTH_FAILED	No	Either stage.
Upload, other 4xx (incl. 403)	MEDIA_UPLOAD_FAILED	No	X detail text in Message.
Upload, 5xx	MEDIA_UPLOAD_FAILED	Yes	
Upload, connection error	NETWORK_ERROR	Yes	Includes connect timeout.
Post, other 4xx	POST_CREATE_FAILED	No	Request was rejected; nothing created.
Post, 5xx	POST_RESULT_UNKNOWN	No	Server may have processed it.
Post, connect timeout	NETWORK_ERROR	Yes	Request was never sent.
Post, any other connection error or unreadable 2xx body	POST_RESULT_UNKNOWN	No	Request may have been sent.

4.5 Credentials file

`X_Keys.txt` uses plain `NAME=VALUE` lines (blank lines and `#` comments allowed). Required names: `API_KEY`, `API_SECRET`, `ACCESS_TOKEN`, `ACCESS_TOKEN_SECRET`. Python reads the path from `KeysFilePath` in X_Config.txt. Missing names are reported by name only, never by value. **[ADAPT]** if your existing X_Keys.txt uses other names: change `REQUIRED_KEYS` and `XBackend.__init__`.

4.6 Mock backend and fault flags

`--fault`	What the mock does	Expected PS1 outcome
upload_fail_2	ERROR MEDIA_UPLOAD_FAILED on MediaId 2 at 61%	ERROR MEDIA_UPLOAD_FAILED
crash_mid	process exits (code 3) after PROGRESS 37, no terminal event	PYTHON_EXITED_UNEXPECTEDLY
garbage_stdout	writes `Uploading...` to stdout	PROTOCOL_INVALID_STDOUT_JSON
wrong_request_id	one PROGRESS with another RequestId	PROTOCOL_REQUEST_ID_MISMATCH
stderr_noise	writes diagnostics to stderr during a valid run	SUCCESS; lines in the log
hang	sleeps after PROGRESS 25	OPERATION_TIMEOUT (idle)
skip_media_complete	omits MEDIA_COMPLETE for media 1	PROTOCOL_SEQUENCE_ERROR
post_unknown	ERROR POST_RESULT_UNKNOWN after POSTING	unknown-outcome flow
extra_after_terminal	writes one more line after SUCCESS	SUCCESS; extra line ignored

The mock emits `Percent` 0, 25, 37, 50, 75, 100 per file and XMediaId `1NN000000000000000` where NN is the zero-padded MediaId.

5. PowerShell instructions (XPoster.ps1)

5.1 Structure

Region of the script	Contains
Constants	Config path, protocol version, timeouts, `ExtraPythonArgs` (dev).
C# runner	`XPosterRunner`: starts Python, writes the request, collects stdout/stderr/exit in a thread-safe queue.
Helpers, ReadConfig, logging adapter	`Get-Prop`, `ReadConfig`, `WriteDebugOutput`, `WriteProtocolLog`.
Application state	`New-AppState`, `Reset-OperationState`.
Request, export	`New-PythonPostRequest`, `Test-PythonPostRequest`, `ExportPostRecord`, `WriteHistoryRow`.
Operation control	`RunPythonEngine`, `PumpRunner`, `StopPostingOperation`, `BeginDrain`, `FinishOperation`.
Router	`HandlePythonResponse`, `HandleProgressEvent`, `HandleMediaCompleteEvent`, `HandleStderrLine`, `HandleProcessExit`.
Results	`CompletePostWorkflow`, `HandlePostingError`, `Get-ErrorAdvice`.
GuiController	`DetermineStartupMode`, `InitializeGui`, `AttachMedia`, `RemoveMedia`, `UpdatePostButtonState`, `UpdateGui`, `UpdateUploadProgress`, `StartPost`.
MainController	Top-level code: ReadConfig, dot-source DebugLog.ps1, DetermineStartupMode, InitializeGui, show the form.

5.2 Process handling and threading (the key design decision)

`Process.OutputDataReceived` fires on a thread-pool thread. A PowerShell script block attached to that event is not guaranteed to run in a usable runspace: it may lose access to script functions and variables, fail silently, or touch WinForms controls from the wrong thread. This design therefore never attaches PowerShell code to process events.

1. A small C# class (`XPosterRunner`, compiled with `Add-Type`) owns the `Process`. Its event handlers are compiled C# that only enqueue `RunnerItem`s (OUT, ERR, EXIT) into a `ConcurrentQueue`.
2. A WinForms `Timer` (100 ms) on the GUI thread calls `PumpRunner`, which drains the queue and calls the router. Everything that touches state or controls therefore runs on the GUI thread; no `Invoke` is needed and the earlier `InvokeGuiUpdate` wrapper is removed.
3. The pump also runs the watchdogs (section 3.6) and finalizes the operation when draining ends.
4. `PumpRunner` is guarded against re-entry, and the timer is stopped before any modal dialog, because a message box would otherwise let the timer fire again inside the dialog.

Encoding. `ProcessStartInfo.StandardInputEncoding` is not available on the .NET Framework that Windows PowerShell 5.1 runs on, so the request is written as UTF-8 bytes straight to the stdin stream. stdout and stderr are decoded as UTF-8 (`StandardOutputEncoding` exists everywhere), and because Python's events are pure ASCII the stdout decoding cannot go wrong.

5.3 StartPost sequence

1. Ignore the click if a post is already running or the post is empty. If `UnknownOutcomePending` is set, ask for confirmation first.
2. Set `PostInProgress`, lock the inputs.
3. Generate `RequestId`, call `ExportPostRecord` (text file, media copies, Pending history row).
4. Build the request from the exported file paths and validate it locally (`Test-PythonPostRequest`).
5. `Reset-OperationState`, then `RunPythonEngine`: serialize with `ConvertTo-Json -Depth 6 -Compress`, start the runner, start the pump timer. A start failure becomes `PYTHON_START_FAILED`.
6. Everything after this arrives through the pump. `StartPost` returns immediately; the GUI stays alive.

5.4 Rules the router MUST follow

- Parse each OUT line with `ConvertFrom-Json`; an unparsable non-blank line is `PROTOCOL_INVALID_STDOUT_JSON` and kills Python. Blank lines are ignored.
- Check `RequestId` on every event (null allowed only for ERROR), then the state-machine table.
- Use `Get-Prop` for every optional or untrusted field (the script runs in strict mode).
- Apply `Percent` clamped to 0 to 100. Store `XMediaId` as a string.
- stderr lines go to the log and a 20-line tail buffer, never to the router.
- Success: set `PublishedPostId` and `PublishedPostUrl`, append a Published history row, show the URL, clear the text and media. Failure: append a Failed row, show the code, message and advice, **keep** the text and media so the user can retry.

5.5 GUI rules

- Post is enabled only when (text is not blank or at least one media is attached) and no post is running and startup checks passed.
- While a post runs: text is read-only; Attach and Remove are disabled; the progress label reads "Uploading file N of M: name (P%)" and switches to the current file; during POSTING it reads "Publishing post...".
- AttachMedia enforces: at most 4 files, supported extensions, and a GIF or video only as the sole item.
- Closing the window while a post runs is cancelled with a message (killing mid-post could create an unknown outcome).

5.6 Logging

Everything is logged through `WriteDebugOutput`, which wraps the DebugLog.ps1 function named in DebugLogReadMe.txt. Protocol lines use `WriteProtocolLog`: `[RequestId=...] PY->PS PROGRESS MediaId=2 FileIndex=2 Percent=37`. **[ADAPT]** the body of `WriteDebugOutput`. Verify in Phase 5 that a log file is actually produced; the default body falls back to `Debug.WriteLine` if the logger call fails, which would otherwise hide a missing logger.

6. Logging and security rules

Never log or emit	Safe to log
API keys and secrets, access/refresh tokens, Authorization headers, the contents of X_Keys.txt, raw request or response bodies, the post text, full local file paths.	RequestId, direction, event name, MediaId, FileIndex, Percent, error Code, HttpStatus, text length, file names, exit codes, stderr diagnostics.

- Credentials never enter the protocol and are never passed on the command line. The only argument that identifies a file is `--config`.
- Launch Python directly with explicit arguments; never build a shell command string.
- `Message` text for X errors is the X `detail/title`, trimmed to 160 characters.

7. Test plan

7.1 Running the conformance harness

`python xposter_conformance.py` (Appendix C) starts the engine in `--fake` mode and plays the PowerShell role with the same state machine, timeouts shortened to 3 s. It must print PASS for all tests. Run it on the target machine before touching the GUI.

7.2 Test matrix

Test	Scenario	Expected result
A	Text only	READY, TEXT_COMPLETE, POSTING, SUCCESS
B	One media, empty text	TEXT_COMPLETE with TextPresent=false; 6 PROGRESS; MEDIA_COMPLETE; SUCCESS
C	Four media	MediaId 1 to 4 in order, four XMediaIds stored, SUCCESS
D	Error during media 2 (`upload_fail_2`)	ERROR MEDIA_UPLOAD_FAILED, MediaId 2, no POSTING, exit code 1
E	Crash without ERROR (`crash_mid`)	PYTHON_EXITED_UNEXPECTEDLY, exit code 3
F	Garbage on stdout	PROTOCOL_INVALID_STDOUT_JSON
G	Wrong RequestId	PROTOCOL_REQUEST_ID_MISMATCH
H	Blank text, no media	ERROR EMPTY_POST as the only event
I	Unicode and multi-line text	SUCCESS; CharacterCount matches; stdout is pure ASCII
J	stderr noise during a valid run	SUCCESS; stderr lines logged, not routed
K	Silence (hang)	OPERATION_TIMEOUT
L	Missing MEDIA_COMPLETE	PROTOCOL_SEQUENCE_ERROR
M	`post_unknown`	POST_RESULT_UNKNOWN; unknown-outcome confirmation on the next Post
N	Line after SUCCESS	SUCCESS stands; extra line ignored
O1-O7	Bad requests	UNSUPPORTED_PROTOCOL_VERSION; MALFORMED_REQUEST_JSON (RequestId null); MEDIA_FILE_NOT_FOUND; MEDIA_COUNT_INVALID (5 files); MEDIA_COMBINATION_INVALID; TEXT_TOO_LONG; INVALID_REQUEST

7.3 PowerShell-side tests

Set `$script:ExtraPythonArgs = '--fake'` and run tests A to C through the GUI; then set `'--fake --fault <name>'` for D to N. For each, confirm: the GUI keeps repainting while uploading, the final dialog shows the expected code, inputs are preserved after errors and cleared after success, the Post button returns, and the log shows one RequestId from start to finish with no secrets. Also test: close the window during a post (must be refused), and click Post twice quickly (second click ignored). For `post_unknown`, confirm the next Post asks for confirmation.

7.4 Real-API smoke test (Phase 6)

1. Remove `--fake`. Post a text-only message. Confirm `SUCCESS` and open `PostUrl`.
2. Post one small JPEG with text. Confirm PROGRESS reaches 100 and the image appears.
3. Post four images. Confirm progress switches file 1 to 4.
4. Rename a selected file before pressing Post to confirm `MEDIA_FILE_NOT_FOUND`/export error handling.
5. Temporarily break one credential value to confirm `AUTH_FAILED` with no secret in the log.
6. If the chunked path is rejected for images, only `XBackend.upload_media` needs changing (for example to the single-request upload for small images); the protocol is unaffected.

7.5 Final acceptance checklist

Check	Pass when
Locked names	All names in section 1.2 appear unchanged in both files.
Protocol purity	Python stdout contains only protocol lines in every test.
Responsiveness	GUI never freezes, including during timeout and crash tests.
Recovery	No automatic retry exists anywhere; unknown outcomes require confirmation.
Security	No secret, post text or full path in any log; Python started without a shell.
Traceability	One RequestId identifies request, events and history rows.
Real posts	Smoke tests 1 to 3 passed on the live account.

8. Decision log: what changed from the earlier draft

ID	Finding in earlier draft	Decision
D1	The PDF clipped long JSON lines and wrapped them with stray spaces, so examples could not be copied safely.	Wire examples are real captured lines shown wrapped per record; code is limited to 110 columns; full source is in the appendices.
D2	A footnote number fused to a config key (`2DebugLogReadMePath`).	Documented as a rendering artifact (section 1.2).
D3	Process events were handled by PowerShell script blocks (wrong thread / runspace risk) and `InvokeGuiUpdate` was undefined.	C# queue runner plus GUI-thread timer pump (section 5.2).
D4	`StandardInputEncoding` was marked "where supported"; it is not on .NET Framework.	Write UTF-8 bytes to the stdin stream; Python reads bytes.
D5	Python used `ensure_ascii=False`, making stdout encoding-dependent.	ASCII-only events, written as bytes with a bare LF.
D6	READY meant only "request parsed"; file and credential failures could follow it.	READY now means fully validated and ready to execute (section 3.3).
D7	Python had no way to find `KeysFilePath` (config was locked).	`--config` argument carries the config path only.
D8	No state machine behind `PROTOCOL_SEQUENCE_ERROR`.	Explicit phase table (section 3.4).
D9	`OPERATION_TIMEOUT` and exit handling had no numbers or ordering.	Timeouts (3.6); exit item queued after stream EOF (3.5).
D10	An ERROR for an unreadable request has `RequestId=null`, which a strict ID check would misreport as a mismatch.	Router accepts null RequestId for ERROR only; tested (O2).
D11	X media rules were missing.	Up to 4 images, or one GIF, or one video; size caps; new codes MEDIA_TYPE_UNSUPPORTED, MEDIA_TOO_LARGE, MEDIA_COMBINATION_INVALID, TEXT_TOO_LONG.
D12	`upload_media`/`create_post` were stubs.	Real v2 flow (initialize, append, finalize, status, create post) with full error mapping (4.4).
D13	Unknown-outcome handling was stated but not enforceable.	`UnknownOutcomePending` flag and a confirmation before the next Post.
D14	Closing the window mid-post could orphan Python or create an unknown outcome.	FormClosing is cancelled while a post runs.
D15	The fault tests were descriptions only.	`--fake --fault` flags and a runnable conformance harness.
D16	Idle timeout would false-trigger while X processes video.	Heartbeat PROGRESS during processing polls.

9. Review history

This version was reviewed three times, as if each revision were uploaded again by a different reviewer. Findings and the fix applied to the document or code follow.

Review 1: fidelity to the earlier draft and layout

Reviewed as if uploaded cold by someone comparing it against the earlier PDF and the conversation notes, then rendering every page. An automated check confirmed that all 70 locked names (events, request fields, config keys, GUI state names, function names, error codes) appear unchanged.

Finding	Fix applied
Captions such as "Same request, pretty-printed" and "Upload failure on media 2" were stranded at the bottom of a page, away from the block they describe.	Captions now stay with the following block.
Content from the earlier draft had been dropped: the rejected alternatives (key=value text, sockets), the parallel-upload note, and the media-only and empty-post examples.	Added section 3.10 and two more captured transcripts (media-only, empty post).
The earlier draft's wire examples could only be trusted visually.	All transcripts are now regenerated from the running engine every time the PDF is built, so they cannot drift from the code.

> **Result**
>
> No locked name changed. Layout orphans fixed; omitted material restored.

Review 2: execution and failure review

Reviewed as if uploaded by an engineer who runs the Python code and reads the PowerShell line by line looking for ways it can hang, crash or lie. Python was executed through the conformance harness; PowerShell was read only, because no PowerShell runtime was available.

Finding	Fix applied
Running the harness failed test O2: an ERROR for an unreadable request carries RequestId null, and the strict ID check reported it as a mismatch.	Router and harness accept a null RequestId for ERROR only (decision D10).
Harness test B had a loose assertion that could pass without checking anything.	Assertion made exact (6 PROGRESS, event tail, XMediaId value).
PowerShell: if Python failed to start, the pump timer was never running, so the operation never finished and the GUI stayed locked.	Timer starts before launch; a start failure marks the process as already exited so draining ends at once.
PowerShell: the C# exit-watcher thread could throw after the runner was disposed (grace-period kill), and an unhandled background-thread exception would take down the whole PowerShell host.	Watcher uses a local reference inside try/catch; Dispose only kills the process.
PowerShell: sequence errors used a script block that depended on dynamic variable scoping across three functions.	Replaced by the function `FailSequence`.
PowerShell: the startup check required X_Keys.txt even in mock mode, which would have blocked Phase 3 before any key exists.	Keys file is only required when `--fake` is absent.
PowerShell: dot-sourcing DebugLog.ps1 straight from a property expression, and testing a flag with `Test-Path variable:`.	Path copied to a variable first; the flag is a plain boolean.
No test covered non-ASCII file names, which matter for this installation (Hebrew names).	Added test P: Hebrew file name and text round trip.
Thirteen PowerShell lines exceeded 100 columns.	Code width limit set to 110 columns and enforced by the PDF build (the build fails on a longer line).

> **Result**
>
> Python conformance 22 of 22 pass. PowerShell remains unexecuted and the document says so.

Review 3: handoff completeness and print quality

Reviewed as if uploaded by a developer who has never seen the project and must build it without asking questions, reading the rendered pages rather than the source.

Finding	Fix applied
Nothing said how to start the application, install the real-engine packages, or what X_Keys.txt looks like, so a newcomer would have to guess.	Added section 1.5: the full X_Config.txt, an X_Keys.txt template, the pip command, the launch command (with -STA), the Read-and-write app permission, and the developer switch.
The earlier draft's sequence diagram was missing, leaving the order of PS1, Python and X implicit.	Added a plain-text sequence diagram to section 3.
Code listings were cut at fixed line counts, sometimes in the middle of a function, and large blocks left half-empty pages.	Listings now break at blank lines in smaller pieces, which fill pages and cut between functions.
The table of contents ran onto a second page with large gaps.	Tighter spacing.
It was not stated that the X_Config.txt log-path placeholder is not for XPoster.ps1 to replace.	Stated in 1.5.

Result

A newcomer now has setup, launch, sequence, test and recovery information in one place. Remaining known limits are listed in the assumptions table.

Appendix A. XPoster.py (complete)

Deploy to `Z:\InsPro\Tools\Py\XPoster.py`. Executed and passing Appendix C in the authoring environment (mock mode). The real X backend could not be run there (no network access) and must pass the section 7.4 smoke test.

```
#!/usr/bin/env python3
"""XPoster.py - XPoster protocol v1 engine (Python side).

Deploy to: Z:\\InsPro\\Tools\\Py\\XPoster.py
Run as:    python.exe -u XPoster.py --config "<path to X_Config.txt>"
Dev only:  add --fake [--fault NAME] to run without the X API.

stdin  : exactly one compact JSON request line (UTF-8)
stdout : protocol events only, one compact ASCII JSON object per line
stderr : human-readable diagnostics only
"""
import argparse
import json
import os
import sys
import time

PROTOCOL_VERSION = 1
MAX_MEDIA = 4
MAX_TEXT_CHARS = 280            # set to 25000 for a Premium account
MAX_IMAGE_BYTES = 5 * 1024 * 1024
MAX_GIF_BYTES = 15 * 1024 * 1024
MAX_VIDEO_BYTES = 512 * 1024 * 1024   # conservative application cap
CHUNK_BYTES = 2 * 1024 * 1024         # X allows up to 5 MB per APPEND
HTTP_TIMEOUT = (10, 60)               # (connect, read) seconds
PROCESSING_TIMEOUT_SEC = 600
API_BASE = "https://api.x.com"

MEDIA_TYPES = {
    ".jpg": ("image/jpeg", "tweet_image", MAX_IMAGE_BYTES),
    ".jpeg": ("image/jpeg", "tweet_image", MAX_IMAGE_BYTES),
    ".png": ("image/png", "tweet_image", MAX_IMAGE_BYTES),
    ".webp": ("image/webp", "tweet_image", MAX_IMAGE_BYTES),
    ".gif": ("image/gif", "tweet_gif", MAX_GIF_BYTES),
    ".mp4": ("video/mp4", "tweet_video", MAX_VIDEO_BYTES),
    ".mov": ("video/quicktime", "tweet_video", MAX_VIDEO_BYTES),
}

_terminal_sent = False
FAULT = None
```

```
# ------------------------------------------------------------------ output
def configure_streams():
    """stderr must never raise on odd characters; stdout is written as bytes."""
    try:
        sys.stderr.reconfigure(encoding="utf-8", errors="backslashreplace")
    except Exception:
        pass


def debug(message):
    """Diagnostics go to stderr only. Never print() to stdout."""
    try:
        sys.stderr.write(str(message) + "\n")
        sys.stderr.flush()
    except Exception:
        pass


def write_stdout_line(line):
    """Write one protocol line as ASCII bytes with a bare LF, then flush."""
    sys.stdout.buffer.write(line.encode("ascii") + b"\n")
    sys.stdout.buffer.flush()


def emit_event(event, request_id, **fields):
    """The ONLY function allowed to write protocol events to stdout."""
    global _terminal_sent
    if _terminal_sent:
        debug("suppressed %s: a terminal event was already sent" % event)
        return
    payload = {"Event": event, "RequestId": request_id}
    payload.update(fields)
    line = json.dumps(payload, ensure_ascii=True,
                      separators=(",", ":"), allow_nan=False)
    write_stdout_line(line)
    if event in ("SUCCESS", "ERROR"):
        _terminal_sent = True
```

```
class KnownProtocolError(Exception):
    def __init__(self, code, stage, message, retryable=False, media_id=None,
                 file_index=None, http_status=None, diagnostic=None):
        Exception.__init__(self, message)
        self.code = code
        self.stage = stage
        self.safe_message = message
        self.retryable = retryable
        self.media_id = media_id
        self.file_index = file_index
        self.http_status = http_status
        self.diagnostic = diagnostic or message


def emit_error(request_id, exc):
    extra = {}
    if exc.media_id is not None:
        extra["MediaId"] = exc.media_id
    if exc.file_index is not None:
        extra["FileIndex"] = exc.file_index
    if exc.http_status is not None:
        extra["HttpStatus"] = exc.http_status
    emit_event("ERROR", request_id, Code=exc.code, Stage=exc.stage,
               Message=exc.safe_message, Retryable=bool(exc.retryable), **extra)
    debug("ERROR %s: %s" % (exc.code, exc.diagnostic))


class MediaItem(object):
    def __init__(self, media_id, file_index, file_name, path, mime, category, size):
        self.media_id = media_id
        self.file_index = file_index
        self.file_name = file_name
        self.path = path
        self.mime = mime
        self.category = category
        self.size = size


class ProgressReporter(object):
    """Emits PROGRESS only when the percent changes (or on heartbeat)."""
```

```
    def __init__(self, request_id, item, total_files):
        self.request_id = request_id
        self.item = item
        self.total_files = total_files
        self.last = None

    def emit(self, percent, heartbeat=False):
        percent = max(0, min(100, int(percent)))
        if percent == self.last and not heartbeat:
            return
        self.last = percent
        emit_event("PROGRESS", self.request_id, MediaId=self.item.media_id,
                   FileIndex=self.item.file_index, TotalFiles=self.total_files,
                   Percent=percent, FileName=self.item.file_name)


# ---------------------------------------------------- request handling
def parse_request(raw):
    try:
        text = raw.decode("utf-8-sig")
    except UnicodeDecodeError as exc:
        raise KnownProtocolError("MALFORMED_REQUEST_JSON", "RequestValidation",
                                 "Request is not valid UTF-8.", diagnostic=str(exc))
    if not text.strip():
        raise KnownProtocolError("MALFORMED_REQUEST_JSON", "RequestValidation",
                                 "No request received on stdin.")
    try:
        req = json.loads(text)
    except ValueError as exc:
        raise KnownProtocolError("MALFORMED_REQUEST_JSON", "RequestValidation",
                                 "Request is not valid JSON.", diagnostic=str(exc))
    if not isinstance(req, dict):
        raise KnownProtocolError("MALFORMED_REQUEST_JSON", "RequestValidation",
                                 "Request must be a JSON object.")
    return req


def extract_request_id(req):
    value = req.get("RequestId")
    return value if isinstance(value, str) and value else None
```

```
def _is_int(value):
    return type(value) is int


def _bad(message):
    return KnownProtocolError("INVALID_REQUEST", "RequestValidation", message)


def validate_request(req):
    """Return (text, [MediaItem]) or raise KnownProtocolError. Local checks only."""
    if req.get("ProtocolVersion") != PROTOCOL_VERSION or not _is_int(req.get("ProtocolVersion")):
        raise KnownProtocolError("UNSUPPORTED_PROTOCOL_VERSION", "RequestValidation",
                                 "Protocol version %r is not supported." % (req.get("ProtocolVersion"),))
    if extract_request_id(req) is None:
        raise _bad("RequestId must be a non-empty string.")
    text = req.get("Text")
    if not isinstance(text, str):
        raise _bad("Text must be a string.")
    total = req.get("TotalFiles")
    media = req.get("Media")
    if not _is_int(total) or total < 0 or total > MAX_MEDIA:
        raise KnownProtocolError("MEDIA_COUNT_INVALID", "RequestValidation",
                                 "TotalFiles must be an integer from 0 to %d." % MAX_MEDIA)
    if not isinstance(media, list):
        raise _bad("Media must be an array.")
    if len(media) != total:
        raise KnownProtocolError("MEDIA_COUNT_INVALID", "RequestValidation",
                                 "Media length does not equal TotalFiles.")
    items, seen_ids = [], set()
    for position, entry in enumerate(media, start=1):
        if not isinstance(entry, dict):
            raise _bad("Media entry %d is not an object." % position)
        mid, idx = entry.get("MediaId"), entry.get("FileIndex")
        name, path = entry.get("FileName"), entry.get("FilePath")
        if not _is_int(mid) or not _is_int(idx):
            raise _bad("Media entry %d needs integer MediaId and FileIndex." % position)
        if not isinstance(name, str) or not isinstance(path, str) or not name or not path:
            raise _bad("Media entry %d needs FileName and FilePath strings." % position)
        if mid in seen_ids:
            raise _bad("MediaId %d is duplicated." % mid)
        if idx != position:
            raise _bad("FileIndex must be sequential starting at 1.")
        seen_ids.add(mid)
        items.append(_check_file(mid, idx, name, path))
```

```
    if text.strip() == "" and total == 0:
        raise KnownProtocolError("EMPTY_POST", "RequestValidation",
                                 "Post must contain text or at least one media item.")
    if len(items) > 1 and any(i.category != "tweet_image" for i in items):
        raise KnownProtocolError("MEDIA_COMBINATION_INVALID", "RequestValidation",
                                 "A GIF or video must be the only media item in a post.")
    return prepare_text(text), items


def _check_file(mid, idx, name, path):
    def fail(code, message, diag=None):
        return KnownProtocolError(code, "RequestValidation", message,
                                  media_id=mid, file_index=idx, diagnostic=diag)
    if not os.path.isabs(path):
        raise fail("INVALID_REQUEST", "FilePath must be absolute.")
    if not os.path.isfile(path):
        raise fail("MEDIA_FILE_NOT_FOUND", "Media file not found: %s" % name, path)
    try:
        size = os.path.getsize(path)
        with open(path, "rb") as handle:
            handle.read(1)
    except OSError as exc:
        raise fail("MEDIA_FILE_UNREADABLE", "Media file cannot be read: %s" % name, str(exc))
    if size == 0:
        raise fail("MEDIA_FILE_UNREADABLE", "Media file is empty: %s" % name)
    ext = os.path.splitext(path)[1].lower()
    if ext not in MEDIA_TYPES:
        raise fail("MEDIA_TYPE_UNSUPPORTED", "Unsupported media type: %s" % name)
    mime, category, limit = MEDIA_TYPES[ext]
    if size > limit:
        raise fail("MEDIA_TOO_LARGE", "Media file is too large: %s" % name)
    return MediaItem(mid, idx, name, path, mime, category, size)


def prepare_text(text):
    """Normalise line endings only. Never trim or otherwise alter user text."""
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    if len(text) > MAX_TEXT_CHARS:
        raise KnownProtocolError("TEXT_TOO_LONG", "RequestValidation",
                                 "Text is %d characters; the limit is %d."
                                 % (len(text), MAX_TEXT_CHARS))
    return text
```

```
# ------------------------------------------------------ config and keys
def read_key_value_file(path, what):
    values = {}
    try:
        with open(path, "r", encoding="utf-8-sig") as handle:
            for line in handle:
                line = line.strip()
                if not line or line.startswith("#") or "=" not in line:
                    continue
                key, value = line.split("=", 1)
                values[key.strip()] = value.strip()
    except OSError as exc:
        code = "CONFIG_ERROR" if what == "config" else "CREDENTIAL_ERROR"
        raise KnownProtocolError(code, "Startup", "Cannot read %s file." % what,
                                 diagnostic=str(exc))
    return values


REQUIRED_KEYS = ("API_KEY", "API_SECRET", "ACCESS_TOKEN", "ACCESS_TOKEN_SECRET")


def load_keys(config_path):
    if not config_path:
        raise KnownProtocolError("CONFIG_ERROR", "Startup", "--config argument is missing.")
    config = read_key_value_file(config_path, "config")
    keys_path = config.get("KeysFilePath")
    if not keys_path:
        raise KnownProtocolError("CONFIG_ERROR", "Startup", "KeysFilePath is missing in config.")
    keys = read_key_value_file(keys_path, "credentials")
    missing = [name for name in REQUIRED_KEYS if not keys.get(name)]
    if missing:
        raise KnownProtocolError("CREDENTIAL_ERROR", "Startup",
                                 "Credentials missing: " + ", ".join(missing))
    return keys   # values are secrets: never log or emit them


# ---------------------------------------------------------- backends
class PostResult(object):
    def __init__(self, post_id, post_url):
        self.post_id = post_id
        self.post_url = post_url
```

```
class FakeBackend(object):
    """Dev/test backend. Same events as the real one, no network."""

    def __init__(self, request_id):
        self.request_id = request_id

    def upload_media(self, item, progress):
        if FAULT == "garbage_stdout" and item.media_id == 1:
            sys.stdout.buffer.write(b"Uploading...\n")
            sys.stdout.buffer.flush()
        if FAULT == "stderr_noise":
            debug("[fake] upload retry attempt failed: timeout")
        for percent in (0, 25, 37, 50, 75, 100):
            progress.emit(percent)
            if percent == 25 and item.media_id == 1:
                if FAULT == "hang":
                    time.sleep(3600)
                if FAULT == "wrong_request_id":
                    emit_wrong_request_id(item)
            if percent == 37 and item.media_id == 1 and FAULT == "crash_mid":
                os._exit(3)
            if percent == 50 and item.media_id == 2 and FAULT == "upload_fail_2":
                progress.emit(61)
                raise KnownProtocolError("MEDIA_UPLOAD_FAILED", "MediaUpload",
                                         "Media upload failed.", retryable=True,
                                         media_id=item.media_id, file_index=item.file_index)
            time.sleep(0.02)
        return "1%02d000000000000000" % item.media_id

    def create_post(self, text, media_ids):
        if FAULT == "post_unknown":
            raise KnownProtocolError("POST_RESULT_UNKNOWN", "PostCreate",
                                     "Connection lost after the post request was sent.")
        return PostResult("9990000000000000001", "https://x.com/i/web/status/9990000000000000001")


def emit_wrong_request_id(item):
    line = json.dumps({"Event": "PROGRESS", "RequestId": "STALE_REQUEST", "MediaId": item.media_id,
                       "FileIndex": item.file_index, "TotalFiles": 1, "Percent": 26,
                       "FileName": item.file_name}, separators=(",", ":"))
    write_stdout_line(line)
```

```
class XBackend(object):
    """Real X API v2 backend: chunked media upload, then POST /2/tweets."""

    def __init__(self, keys):
        try:
            import requests
            from requests_oauthlib import OAuth1Session
        except ImportError as exc:
            raise KnownProtocolError(
                "CONFIG_ERROR", "Startup",
                "Python packages 'requests' and 'requests-oauthlib' are required.",
                diagnostic=str(exc))
        self._requests = requests
        self._session = OAuth1Session(
            keys["API_KEY"], client_secret=keys["API_SECRET"],
            resource_owner_key=keys["ACCESS_TOKEN"],
            resource_owner_secret=keys["ACCESS_TOKEN_SECRET"])

    # -- error mapping ------------------------------------------------
    @staticmethod
    def _detail(resp):
        try:
            body = resp.json()
            text = str(body.get("detail") or body.get("title") or "")
        except Exception:
            text = ""
        return " ".join(text.split())[:160]
```

```
    def _call(self, method, path, stage, item=None, **kwargs):
        ids = {}
        if item is not None:
            ids = {"media_id": item.media_id, "file_index": item.file_index}
        try:
            resp = self._session.request(method, API_BASE + path,
                                         timeout=HTTP_TIMEOUT, **kwargs)
        except self._requests.exceptions.ConnectTimeout as exc:
            raise KnownProtocolError("NETWORK_ERROR", stage, "Could not connect to X.",
                                     retryable=True, diagnostic=repr(exc), **ids)
        except self._requests.exceptions.RequestException as exc:
            if stage == "PostCreate":
                raise KnownProtocolError(
                    "POST_RESULT_UNKNOWN", stage,
                    "Connection failed after the post request was sent; check X before retrying.",
                    diagnostic=repr(exc))
            raise KnownProtocolError("NETWORK_ERROR", stage, "Network error talking to X.",
                                     retryable=True, diagnostic=repr(exc), **ids)
        if 200 <= resp.status_code < 300:
            return resp
        status, detail = resp.status_code, self._detail(resp)
        message = "X API returned HTTP %d. %s" % (status, detail)
        if status == 429:
            raise KnownProtocolError("RATE_LIMITED", stage, message, retryable=True,
                                     http_status=status, **ids)
        if status == 401:
            raise KnownProtocolError("AUTH_FAILED", stage, message, http_status=status, **ids)
        if stage == "PostCreate":
            code = "POST_RESULT_UNKNOWN" if status >= 500 else "POST_CREATE_FAILED"
            raise KnownProtocolError(code, stage, message, http_status=status)
        raise KnownProtocolError("MEDIA_UPLOAD_FAILED", stage, message,
                                 retryable=status >= 500, http_status=status, **ids)

    def _json(self, resp, stage, item=None):
        try:
            return resp.json()["data"]
        except Exception:
            if stage == "PostCreate":
                raise KnownProtocolError("POST_RESULT_UNKNOWN", stage,
                                         "Unreadable response to the post request.")
            raise KnownProtocolError("MEDIA_UPLOAD_FAILED", stage, "Unexpected response from X.",
                                     media_id=item.media_id, file_index=item.file_index)

    # -- operations ------------------------------------------------------
    def upload_media(self, item, progress):
        progress.emit(0)
        init = self._json(self._call(
            "POST", "/2/media/upload/initialize", "MediaUpload", item,
            json={"media_type": item.mime, "total_bytes": item.size,
                  "media_category": item.category}), "MediaUpload", item)
        x_media_id = str(init["id"])
        sent, segment = 0, 0
        try:
            with open(item.path, "rb") as handle:
                while True:
                    chunk = handle.read(CHUNK_BYTES)
                    if not chunk:
                        break
                    self._call("POST", "/2/media/upload/%s/append" % x_media_id,
                               "MediaUpload", item,
                               data={"segment_index": str(segment)},
                               files={"media": (item.file_name, chunk, "application/octet-stream")})
                    sent += len(chunk)
                    segment += 1
                    progress.emit(min(99, sent * 100 // item.size))
        except OSError as exc:
            raise KnownProtocolError("MEDIA_FILE_UNREADABLE", "MediaUpload",
                                     "Media file could not be read during upload.",
                                     media_id=item.media_id, file_index=item.file_index,
                                     diagnostic=str(exc))
        final = self._json(self._call(
            "POST", "/2/media/upload/%s/finalize" % x_media_id, "MediaUpload", item),
            "MediaUpload", item)
        self._wait_processing(x_media_id, final.get("processing_info"), item, progress)
        progress.emit(100)
        return x_media_id
```

```
    def _wait_processing(self, x_media_id, info, item, progress):
        deadline = time.time() + PROCESSING_TIMEOUT_SEC
        while info and info.get("state") in ("pending", "in_progress"):
            if time.time() > deadline:
                raise KnownProtocolError("MEDIA_UPLOAD_FAILED", "MediaUpload",
                                         "Media processing timed out.", retryable=True,
                                         media_id=item.media_id, file_index=item.file_index)
            time.sleep(max(1, min(15, int(info.get("check_after_secs", 1)))))
            progress.emit(99, heartbeat=True)      # keeps the PS1 idle watchdog fed
            data = self._json(self._call(
                "GET", "/2/media/upload", "MediaUpload", item,
                params={"command": "STATUS", "media_id": x_media_id}), "MediaUpload", item)
            info = data.get("processing_info")
        if info and info.get("state") == "failed":
            raise KnownProtocolError("MEDIA_UPLOAD_FAILED", "MediaUpload",
                                     "X could not process the media file.",
                                     media_id=item.media_id, file_index=item.file_index)

    def create_post(self, text, media_ids):
        payload = {}
        if text.strip():
            payload["text"] = text
        if media_ids:
            payload["media"] = {"media_ids": list(media_ids)}
        data = self._json(self._call("POST", "/2/tweets", "PostCreate", json=payload), "PostCreate")
        post_id = str(data["id"])
        return PostResult(post_id, "https://x.com/i/web/status/" + post_id)


# ---------------------------------------------------------------------- main
def parse_args(argv):
    parser = argparse.ArgumentParser(add_help=False)
    parser.add_argument("--config", default=None)
    parser.add_argument("--fake", action="store_true")
    parser.add_argument("--fault", default=None)
    return parser.parse_args(argv)

def run(args):
    global FAULT
    FAULT = args.fault if args.fake else None
    request_id = None
    try:
        req = parse_request(sys.stdin.buffer.readline())
        request_id = extract_request_id(req)
        text, items = validate_request(req)
        backend = FakeBackend(request_id) if args.fake else XBackend(load_keys(args.config))

        emit_event("READY", request_id, ProtocolVersion=PROTOCOL_VERSION)
        emit_event("TEXT_COMPLETE", request_id, TextPresent=text.strip() != "",
                   CharacterCount=len(text))

        x_media_ids = []
        for item in items:
            progress = ProgressReporter(request_id, item, len(items))
            x_media_id = backend.upload_media(item, progress)
            x_media_ids.append(x_media_id)
            if FAULT == "skip_media_complete" and item.media_id == 1:
                continue
            emit_event("MEDIA_COMPLETE", request_id, MediaId=item.media_id,
                       FileIndex=item.file_index, TotalFiles=len(items),
                       FileName=item.file_name, XMediaId=x_media_id)

        emit_event("POSTING", request_id, MediaCount=len(items))
        result = backend.create_post(text, x_media_ids)
        emit_event("SUCCESS", request_id, PostId=result.post_id, PostUrl=result.post_url)
        if FAULT == "extra_after_terminal":
            write_stdout_line('{"Event":"PROGRESS","RequestId":"%s","MediaId":1,"FileIndex":1,'
                              '"TotalFiles":1,"Percent":5,"FileName":"x.jpg"}' % request_id)
        return 0
    except KnownProtocolError as exc:
        emit_error(request_id, exc)
        return 1
    except BrokenPipeError:
        return 1                       # PowerShell is gone; nobody to tell
    except Exception as exc:           # noqa: BLE001 - last-resort guard
        emit_event("ERROR", request_id, Code="INTERNAL_ERROR", Stage="Internal",
                   Message="Unexpected Python engine error.", Retryable=False)
        debug("INTERNAL_ERROR: %r" % (exc,))
        return 1
```

```
def main(argv=None):
    configure_streams()
    return run(parse_args(sys.argv[1:] if argv is None else argv))


if __name__ == "__main__":
    sys.exit(main())
```

Appendix B. XPoster.ps1 (complete)

Deploy to `Z:\InsPro\Tools\Ps1\XPoster.ps1`. This file was reviewed line by line and brace-balanced, but PowerShell was not available in the authoring environment, so it has **not** been executed. Phases 3 and 4 are the first run; expect to fix typos there.

```
#requires -Version 5.1
# Z:\InsPro\Tools\Ps1\XPoster.ps1 - XPoster main controller (GUI, workflow, process lifecycle).
# Requires Windows PowerShell 5.1 or PowerShell 7 on Windows (WinForms).
Set-StrictMode -Version 2.0
$ErrorActionPreference = 'Stop'
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing
[System.Windows.Forms.Application]::EnableVisualStyles()

# ============================ constants (NOT part of X_Config.txt) ============================
$script:ConfigPath       = 'Z:\InsPro\Tools\Data\X_Post\Operational\X_Config.txt'
$script:ProtocolVersion  = 1
$script:MaxMedia         = 4
$script:StartupTimeoutSec = 30      # Python start -> READY
$script:IdleTimeoutSec   = 90       # silence on stdout while a post is running
$script:TotalTimeoutSec  = 1800     # hard cap for one post
$script:DrainGraceSec    = 5        # wait for Python to exit after a terminal event
$script:PumpIntervalMs   = 100
$script:ExtraPythonArgs  = ''       # dev only, e.g. '--fake --fault upload_fail_2'

# ============================ C# process runner (C# 5 compatible) ============================
# Reads stdout/stderr on background threads into a queue. PowerShell never touches a
# process callback; the GUI-thread timer drains the queue instead.
if (-not ('XPosterRunner' -as [type])) {
Add-Type -TypeDefinition @'
using System;
using System.Collections.Concurrent;
using System.Diagnostics;
using System.IO;
using System.Text;
using System.Threading;

public sealed class RunnerItem
{
    public string Kind;      // OUT, ERR or EXIT
    public string Text;
    public int ExitCode;
}

public sealed class XPosterRunner : IDisposable
{
    private readonly ConcurrentQueue<RunnerItem> queue = new ConcurrentQueue<RunnerItem>();
    private Process process;
```

```
    private void Add(string kind, string text, int exitCode)
    {
        RunnerItem item = new RunnerItem();
        item.Kind = kind; item.Text = text; item.ExitCode = exitCode;
        queue.Enqueue(item);
    }

    public void Start(string exe, string arguments, string workDir, string requestJson)
    {
        ProcessStartInfo psi = new ProcessStartInfo();
        psi.FileName = exe;
        psi.Arguments = arguments;
        psi.WorkingDirectory = workDir;
        psi.UseShellExecute = false;
        psi.CreateNoWindow = true;
        psi.RedirectStandardInput = true;
        psi.RedirectStandardOutput = true;
        psi.RedirectStandardError = true;
        psi.StandardOutputEncoding = new UTF8Encoding(false);
        psi.StandardErrorEncoding = new UTF8Encoding(false);
        process = new Process();
        process.StartInfo = psi;
        process.OutputDataReceived += delegate(object s, DataReceivedEventArgs e)
            { if (e.Data != null) { Add("OUT", e.Data, 0); } };
        process.ErrorDataReceived += delegate(object s, DataReceivedEventArgs e)
            { if (e.Data != null) { Add("ERR", e.Data, 0); } };
        process.Start();
        process.BeginOutputReadLine();
        process.BeginErrorReadLine();
        Thread watcher = new Thread(WatchExit);
        watcher.IsBackground = true;
        watcher.Start();
        try
        {
            byte[] bytes = new UTF8Encoding(false).GetBytes(requestJson + "\n");
            Stream stdin = process.StandardInput.BaseStream;
            stdin.Write(bytes, 0, bytes.Length);
            stdin.Flush();
            process.StandardInput.Close();
        }
        catch (IOException ex) { Add("ERR", "stdin write failed: " + ex.Message, 0); }
    }
```

```
    private void WatchExit()
    {
        Process p = process;
        int code = -1;
        try
        {
            p.WaitForExit();              // returns only after stdout and stderr hit EOF
            code = p.ExitCode;
        }
        catch (Exception) { }             // never let a background thread crash the host
        Add("EXIT", "", code);
    }

    public RunnerItem Next()
    {
        RunnerItem item;
        return queue.TryDequeue(out item) ? item : null;
    }

    public void Kill()
    {
        try { if (process != null && !process.HasExited) { process.Kill(); } } catch { }
    }

    public void Dispose()
    {
        Kill();                           // the Process object is left to the GC on purpose
    }
}
'@
}

# ============================ helpers ============================
function Get-Prop {
    param($Object, [string]$Name)
    if ($null -eq $Object) { return $null }
    $p = $Object.PSObject.Properties[$Name]
    if ($null -eq $p) { return $null }
    return $p.Value
}

function Test-IsInt { param($Value) return (($Value -is [int]) -or ($Value -is [long])) }
```

```
function ReadConfig {
    param([string]$Path)
    if (-not (Test-Path -LiteralPath $Path -PathType Leaf)) {
        throw "CONFIG_ERROR: config file not found: $Path"
    }
    $cfg = @{}
    foreach ($line in [System.IO.File]::ReadAllLines($Path, [System.Text.Encoding]::UTF8)) {
        $t = $line.Trim()
        $i = $t.IndexOf('=')
        if ($t.Length -eq 0 -or $t.StartsWith('#') -or $i -lt 1) { continue }
        $cfg[$t.Substring(0, $i).Trim()] = $t.Substring($i + 1).Trim()   # split on FIRST '='
    }
    $required = @('PythonPath', 'PythonScriptPath', 'DataRootPath', 'OperationalPath',
        'TextFilesPath', 'MediaFilesPath', 'ExcelFilePath', 'KeysFilePath',
        'DebugLogReadMePath', 'DebugLogFunctionPath', 'DebugLogMode', 'DebugLogOutputPath')
    $missing = @($required | Where-Object {
        -not $cfg.ContainsKey($_) -or [string]::IsNullOrWhiteSpace($cfg[$_]) })
    if ($missing.Count -gt 0) { throw "CONFIG_ERROR: missing keys: $($missing -join ', ')" }
    return [pscustomobject]$cfg
}

# --- logging adapter. [ADAPT] the body of WriteDebugOutput to the call documented in
# --- DebugLogReadMePath. Everything else in this file logs ONLY through this function.
function WriteDebugOutput {
    param([string]$Message)
    try { Write-DebugLog -Message $Message }
    catch { [System.Diagnostics.Debug]::WriteLine($Message) }
}

function WriteProtocolLog {
    param([string]$Direction, $Message)
    $parts = @("[RequestId=$(Get-Prop $Message 'RequestId')]", $Direction,
               [string](Get-Prop $Message 'Event'))
    foreach ($name in 'MediaId', 'FileIndex', 'Percent', 'Code') {
        $v = Get-Prop $Message $name
        if ($null -ne $v) { $parts += "$name=$v" }
    }
    WriteDebugOutput ($parts -join ' ')      # never log Text, paths, XMediaId or secrets
}

function ShowMessage {
    param([string]$Text, [string]$Title = 'XPoster', [string]$Icon = 'Information')
    [void][System.Windows.Forms.MessageBox]::Show(
        $script:Form, $Text, $Title, 'OK', [System.Windows.Forms.MessageBoxIcon]::$Icon)
}

# ============================ application state ============================
function New-AppState {
    return @{
        PostInProgress = $false; Phase = 'Idle'      # Idle Starting Ready Media Posting Draining
        Request = $null; RequestId = ''; Runner = $null
        NextMediaIndex = 0; MediaIds = @{}
        CurrentFileIndex = 0; TotalFiles = 0; CurrentFileProgress = 0; CurrentFileName = ''
        PublishedPostId = ''; PublishedPostUrl = ''; PythonResult = ''
        Result = $null; ExitSeen = $false; ExitCode = -1
        StderrTail = [System.Collections.Generic.Queue[string]]::new()
        Clock = $null; LastEventMs = 0; DrainStartedMs = 0
        UnknownOutcomePending = $false; ExportRecord = $null
    }
}
$script:AppState = New-AppState
$script:SelectedMedia = [System.Collections.Generic.List[string]]::new()
$script:Pumping = $false
$script:StartupBlocked = $false

function Reset-OperationState {
    param($Request)
    $s = $script:AppState
    $s.Request = $Request; $s.RequestId = $Request.RequestId; $s.Phase = 'Starting'
    $s.NextMediaIndex = 0; $s.MediaIds = @{}; $s.TotalFiles = $Request.TotalFiles
    $s.CurrentFileIndex = 0; $s.CurrentFileProgress = 0; $s.CurrentFileName = ''
    $s.PublishedPostId = ''; $s.PublishedPostUrl = ''; $s.PythonResult = ''
    $s.Result = $null; $s.ExitSeen = $false; $s.ExitCode = -1; $s.StderrTail.Clear()
    $s.Clock = [System.Diagnostics.Stopwatch]::StartNew(); $s.LastEventMs = 0
}
```

```
# ============================ request building ============================
function New-PythonPostRequest {
    param([string]$RequestId, [string]$PostText, $ExportedMedia)
    $media = @()
    $n = 0
    foreach ($m in @($ExportedMedia)) {
        $n++
        $media += [pscustomobject]@{
            MediaId = $n; FileIndex = $n; FileName = $m.FileName; FilePath = $m.FilePath }
    }
    return [pscustomobject]@{
        ProtocolVersion = $script:ProtocolVersion; RequestId = $RequestId; Text = $PostText
        TotalFiles = $media.Count; Media = [object[]]$media }
}

function Test-PythonPostRequest {
    param($Request)          # returns an error string, or $null when valid
    if ([string]::IsNullOrWhiteSpace($Request.Text) -and $Request.TotalFiles -eq 0) {
        return 'Post is empty.' }
    if ($Request.TotalFiles -gt $script:MaxMedia) { return 'Too many media files.' }
    if (@($Request.Media).Count -ne $Request.TotalFiles) { return 'Media count mismatch.' }
    foreach ($m in @($Request.Media)) {
        if (-not (Test-Path -LiteralPath $m.FilePath -PathType Leaf)) {
            return "File not found: $($m.FileName)" }
    }
    return $null
}

function Get-MediaKind {
    param([string]$Path)
    switch ([System.IO.Path]::GetExtension($Path).ToLowerInvariant()) {
        { $_ -in '.jpg', '.jpeg', '.png', '.webp' } { return 'image' }
        '.gif' { return 'gif' }
        { $_ -in '.mp4', '.mov' } { return 'video' }
        default { return $null }
    }
}

# ============================ ExportPostRecord and history ============================
function WriteHistoryRow {
    param([string]$RequestId, [string]$Status, [string]$Detail = '', [int]$TextLength = 0,
          [int]$MediaCount = 0)
    # [ADAPT] replace with the Excel export from the original design if one exists.
    $dir = $script:Config.ExcelFilePath
    [void][System.IO.Directory]::CreateDirectory($dir)
    $file = Join-Path $dir 'PostHistory.csv'
    $stamp = (Get-Date).ToString('yyyy-MM-dd HH:mm:ss')
    $row = ($stamp, $RequestId, $Status, $TextLength, $MediaCount, $Detail | ForEach-Object {
        '"' + ([string]$_ -replace '"', '""') + '"' }) -join ','
    [System.IO.File]::AppendAllText($file, $row + "`r`n", [System.Text.UTF8Encoding]::new($true))
}

function ExportPostRecord {
    param([string]$RequestId, [string]$PostText, [string[]]$SelectedMedia)
    $cfg = $script:Config
    [void][System.IO.Directory]::CreateDirectory($cfg.TextFilesPath)
    [void][System.IO.Directory]::CreateDirectory($cfg.MediaFilesPath)
    $textFile = Join-Path $cfg.TextFilesPath ($RequestId + '.txt')
    [System.IO.File]::WriteAllText($textFile, $PostText, [System.Text.UTF8Encoding]::new($false))
    $exported = @()
    $n = 0
    $root = [System.IO.Path]::GetFullPath($cfg.MediaFilesPath).TrimEnd('\') + '\'
    foreach ($src in $SelectedMedia) {
        $n++
        $name = [System.IO.Path]::GetFileName($src)
        $full = [System.IO.Path]::GetFullPath($src)
        if ($full.StartsWith($root, [System.StringComparison]::OrdinalIgnoreCase)) {
            $dest = $full                                  # already inside MediaFilesPath
        } else {
            $dest = Join-Path $cfg.MediaFilesPath ("{0}_{1}_{2}" -f $RequestId, $n, $name)
            Copy-Item -LiteralPath $full -Destination $dest -Force
        }
        $exported += [pscustomobject]@{ FileName = $name; FilePath = $dest }
    }
    WriteHistoryRow -RequestId $RequestId -Status 'Pending' -TextLength $PostText.Length `
        -MediaCount $exported.Count
    return [pscustomobject]@{ TextFile = $textFile; Media = $exported }
}
```

```
# ============================ Python launch ============================
function RunPythonEngine {
    param($Config, $Request, $AppState)
    $json = $Request | ConvertTo-Json -Depth 6 -Compress
    $pyArgs = '-u "{0}" --config "{1}"' -f $Config.PythonScriptPath, $script:ConfigPath
    if ($script:ExtraPythonArgs) { $pyArgs += ' ' + $script:ExtraPythonArgs }
    $workDir = Split-Path -Parent $Config.PythonScriptPath
    $runner = [XPosterRunner]::new()
    $AppState.Runner = $runner
    $script:PumpTimer.Start()                # the pump finishes every operation, even a failed start
    try { $runner.Start($Config.PythonPath, $pyArgs, $workDir, $json) }
    catch {
        WriteDebugOutput "[PS] PYTHON_START_FAILED: $($_.Exception.Message)"
        $AppState.ExitSeen = $true           # no process exists, so there is nothing to wait for
        StopPostingOperation -Code 'PYTHON_START_FAILED' -Message $_.Exception.Message
        return
    }
    WriteDebugOutput ("[RequestId={0}] PS->PY request sent TotalFiles={1}" -f
        $Request.RequestId, $Request.TotalFiles)
}

# ============================ operation control ============================
function StopPostingOperation {
    # Used for every PS1-generated (synthetic) failure. The protocol is no longer trusted.
    param([string]$Code, [string]$Message)
    $s = $script:AppState
    if ($s.Phase -eq 'Draining' -or $s.Phase -eq 'Idle') { return }
    $unknown = ($s.Phase -eq 'Posting')      # post request may already have reached X
    $s.Result = @{ Kind = 'Error'; Code = $Code; Stage = 'Transport'; Message = $Message
                   Retryable = $false; MediaId = $null; HttpStatus = $null
                   OutcomeUnknown = $unknown; Source = 'PS1' }
    WriteDebugOutput "[RequestId=$($s.RequestId)] PS1 $Code $Message"
    if ($null -ne $s.Runner) { $s.Runner.Kill() }
    BeginDrain
}

function BeginDrain {
    $s = $script:AppState
    $s.Phase = 'Draining'
    $s.DrainStartedMs = $s.Clock.ElapsedMilliseconds
    UpdateGui -GuiState 'Finishing'
}
```

```
function FinishOperation {
    $s = $script:AppState
    $script:PumpTimer.Stop()                # modal dialogs below would otherwise re-enter the pump
    if ($null -ne $s.Runner) { $s.Runner.Dispose(); $s.Runner = $null }
    $s.Phase = 'Idle'
    $s.PostInProgress = $false
    if ($s.Result.Kind -eq 'Success') { CompletePostWorkflow } else { HandlePostingError $s.Result }
    UpdatePostButtonState
}

function PumpRunner {
    if ($script:Pumping) { return }
    $script:Pumping = $true
    try {
        $s = $script:AppState
        if (-not $s.PostInProgress -or $null -eq $s.Runner) { return }
        for ($n = 0; $n -lt 200; $n++) {
            $item = $s.Runner.Next()
            if ($null -eq $item) { break }
            switch ($item.Kind) {
                'OUT'  { HandlePythonResponse -PythonMessage $item.Text -AppState $s }
                'ERR'  { HandleStderrLine -Line $item.Text -AppState $s }
                'EXIT' { HandleProcessExit -ExitCode $item.ExitCode -AppState $s }
            }
            if ($s.Phase -eq 'Idle') { return }
        }
        $now = $s.Clock.ElapsedMilliseconds
        if ($s.Phase -eq 'Draining') {
            if ($s.ExitSeen -or ($now - $s.DrainStartedMs) -gt ($script:DrainGraceSec * 1000)) {
                FinishOperation
            }
        } elseif ($s.Phase -eq 'Starting' -and $now -gt ($script:StartupTimeoutSec * 1000)) {
            StopPostingOperation -Code 'OPERATION_TIMEOUT' -Message 'Python did not send READY in time.'
        } elseif (($now - $s.LastEventMs) -gt ($script:IdleTimeoutSec * 1000)) {
            StopPostingOperation -Code 'OPERATION_TIMEOUT' -Message 'No output from Python (idle).'
        } elseif ($now -gt ($script:TotalTimeoutSec * 1000)) {
            StopPostingOperation -Code 'OPERATION_TIMEOUT' -Message 'Post exceeded the total time limit.'
        }
    }
    finally { $script:Pumping = $false }
}

function HandleStderrLine {
    param([string]$Line, $AppState)
    WriteDebugOutput "[PY-STDERR] $Line"          # diagnostics only, never parsed as protocol
    $AppState.StderrTail.Enqueue($Line)
    while ($AppState.StderrTail.Count -gt 20) { [void]$AppState.StderrTail.Dequeue() }
}

function HandleProcessExit {
    param([int]$ExitCode, $AppState)
    $AppState.ExitSeen = $true
    $AppState.ExitCode = $ExitCode
    WriteDebugOutput "[RequestId=$($AppState.RequestId)] PS1 python exit code $ExitCode"
    if ($AppState.Phase -ne 'Draining' -and $AppState.Phase -ne 'Idle') {
        $tail = $AppState.StderrTail.ToArray() -join ' | '
        StopPostingOperation -Code 'PYTHON_EXITED_UNEXPECTEDLY' -Message `
            "Python exited with code $ExitCode before sending a final result. stderr: $tail"
    }
}

# ============================ protocol router (runs on the GUI thread) ============================
function Test-EventFields {
    param($Message, [string[]]$Required)     # returns an error string or $null
    foreach ($name in $Required) {
        if ($null -eq (Get-Prop $Message $name)) { return "Field '$name' is missing." }
    }
    return $null
}

function FailSequence {
    param([string]$EventName, [string]$Phase)
    StopPostingOperation -Code 'PROTOCOL_SEQUENCE_ERROR' `
        -Message "Event $EventName is not allowed in phase $Phase."
}
```

```
function HandlePythonResponse {
    param([string]$PythonMessage, $AppState)
    if ($AppState.Phase -eq 'Draining' -or $AppState.Phase -eq 'Idle') {
        WriteDebugOutput "[PS] ignored stdout line after terminal event"
        return
    }
    $AppState.LastEventMs = $AppState.Clock.ElapsedMilliseconds     # any line counts as activity
    if ([string]::IsNullOrWhiteSpace($PythonMessage)) { return }     # blank lines are harmless
    try { $message = $PythonMessage | ConvertFrom-Json -ErrorAction Stop }
    catch {
        WriteDebugOutput "[PS] invalid stdout line: $PythonMessage"
        StopPostingOperation -Code 'PROTOCOL_INVALID_STDOUT_JSON' -Message 'stdout is not JSON.'
        return
    }
    $eventName = [string](Get-Prop $message 'Event')
    if ([string]::IsNullOrWhiteSpace($eventName)) {
        StopPostingOperation -Code 'PROTOCOL_UNEXPECTED_EVENT' -Message 'Event field is missing.'
        return
    }
    # Only an ERROR may carry RequestId=null (Python could not read the request).
    $rid = Get-Prop $message 'RequestId'
    if ($rid -ne $AppState.RequestId -and -not ($eventName -eq 'ERROR' -and $null -eq $rid)) {
        StopPostingOperation -Code 'PROTOCOL_REQUEST_ID_MISMATCH' -Message 'Unexpected RequestId.'
        return
    }
    WriteProtocolLog -Direction 'PY->PS' -Message $message
    $phase = $AppState.Phase
    switch ($eventName) {
        'READY' {
            if ($phase -ne 'Starting') { FailSequence $eventName $phase; return }
            $AppState.Phase = 'Ready'; $AppState.PythonResult = 'Ready'
            UpdateGui -GuiState 'Starting'
        }
        'TEXT_COMPLETE' {
            if ($phase -ne 'Ready') { FailSequence $eventName $phase; return }
            $AppState.Phase = 'Media'
            UpdateGui -GuiState 'TextComplete'
        }
        'PROGRESS' { HandleProgressEvent -Message $message -AppState $AppState }
        'MEDIA_COMPLETE' {
            HandleMediaCompleteEvent -Message $message -AppState $AppState
        }
        'POSTING' {
            $bad = Test-EventFields $message @('MediaCount')

            if ($bad) { StopPostingOperation -Code 'PROTOCOL_UNEXPECTED_EVENT' -Message $bad; return }
            if ($phase -ne 'Media' -or $AppState.NextMediaIndex -ne $AppState.TotalFiles -or
                [int]$message.MediaCount -ne $AppState.TotalFiles) { FailSequence $eventName $phase; return }
            $AppState.Phase = 'Posting'
            UpdateGui -GuiState 'Posting'
        }
        'SUCCESS' {
            $bad = Test-EventFields $message @('PostId', 'PostUrl')
            if ($bad) { StopPostingOperation -Code 'PROTOCOL_UNEXPECTED_EVENT' -Message $bad; return }
            if ($phase -ne 'Posting') { FailSequence $eventName $phase; return }
            $AppState.PublishedPostId = [string]$message.PostId
            $AppState.PublishedPostUrl = [string]$message.PostUrl
            $AppState.PythonResult = 'Success'
            $AppState.Result = @{ Kind = 'Success' }
            BeginDrain
        }
        'ERROR' {
            $AppState.PythonResult = 'Error'
            $code = [string](Get-Prop $message 'Code')
            $AppState.Result = @{ Kind = 'Error'; Code = $code
                Stage = [string](Get-Prop $message 'Stage')
                Message = [string](Get-Prop $message 'Message')
                Retryable = [bool](Get-Prop $message 'Retryable')
                MediaId = Get-Prop $message 'MediaId'; HttpStatus = Get-Prop $message 'HttpStatus'
                OutcomeUnknown = ($code -eq 'POST_RESULT_UNKNOWN'); Source = 'Python' }
            BeginDrain
        }
        default {
            StopPostingOperation -Code 'PROTOCOL_UNEXPECTED_EVENT' -Message "Unknown event: $eventName"
        }
    }
}
```

```
function Test-MediaEvent {
    # Common checks for PROGRESS and MEDIA_COMPLETE. Returns $true when the event is acceptable.
    param($Message, $AppState, [string[]]$Required, [string]$EventName)
    $bad = Test-EventFields $Message $Required
    if ($bad) { StopPostingOperation -Code 'PROTOCOL_UNEXPECTED_EVENT' -Message $bad; return $false }
    $expected = $AppState.NextMediaIndex + 1
    $phaseOk = ($AppState.Phase -eq 'Media') -and ($AppState.NextMediaIndex -lt $AppState.TotalFiles)
    if (-not $phaseOk -or -not (Test-IsInt $Message.MediaId) -or
        [int]$Message.MediaId -ne $expected) {
        FailSequence $EventName $AppState.Phase
        return $false
    }
    return $true
}

function HandleProgressEvent {
    param($Message, $AppState)
    $fields = @('MediaId', 'FileIndex', 'TotalFiles', 'Percent', 'FileName')
    if (-not (Test-MediaEvent $Message $AppState $fields 'PROGRESS')) { return }
    $AppState.CurrentFileIndex = [int]$Message.FileIndex
    $AppState.TotalFiles = [int]$Message.TotalFiles
    $AppState.CurrentFileProgress = [Math]::Max(0, [Math]::Min(100, [int]$Message.Percent))
    $AppState.CurrentFileName = [string]$Message.FileName
    UpdateUploadProgress -CurrentFileIndex $AppState.CurrentFileIndex -TotalFiles $AppState.TotalFiles `
        -CurrentFileProgress $AppState.CurrentFileProgress -CurrentFileName $AppState.CurrentFileName
}

function HandleMediaCompleteEvent {
    param($Message, $AppState)
    $fields = @('MediaId', 'FileIndex', 'TotalFiles', 'FileName', 'XMediaId')
    if (-not (Test-MediaEvent $Message $AppState $fields 'MEDIA_COMPLETE')) { return }
    $x = [string]$Message.XMediaId
    if ([string]::IsNullOrWhiteSpace($x)) {
        StopPostingOperation -Code 'PROTOCOL_UNEXPECTED_EVENT' -Message 'XMediaId is empty.'
        return
    }
    $AppState.MediaIds[[int]$Message.MediaId] = $x       # IDs stay strings, never numbers
    $AppState.NextMediaIndex++
    $AppState.CurrentFileProgress = 100
    UpdateUploadProgress -CurrentFileIndex ([int]$Message.FileIndex) `
        -TotalFiles ([int]$Message.TotalFiles) -CurrentFileProgress 100 `
        -CurrentFileName ([string]$Message.FileName)
}

# ============================ result handling ============================
function CompletePostWorkflow {
    $s = $script:AppState
    $s.UnknownOutcomePending = $false
    WriteHistoryRow -RequestId $s.RequestId -Status 'Published' -Detail $s.PublishedPostUrl
    WriteDebugOutput "[RequestId=$($s.RequestId)] PS1 SUCCESS PostId=$($s.PublishedPostId)"
    UpdateGui -GuiState 'Success'
    ShowMessage -Text "Post published.`r`n$($s.PublishedPostUrl)" -Title 'XPoster'
    $script:txtPost.Clear()
    $script:SelectedMedia.Clear()
    $script:lstMedia.Items.Clear()
    UpdateGui -GuiState 'Idle'
}

function Get-ErrorAdvice {
    param([string]$Code)
    switch ($Code) {
        'RATE_LIMITED'         { return 'X rate limit reached. Wait a while, then post again.' }
        'NETWORK_ERROR'        { return 'Network problem. Check the connection, then post again.' }
        'AUTH_FAILED'          { return 'X rejected the credentials. Check X_Keys.txt.' }
        'CREDENTIAL_ERROR'     { return 'Credentials are missing or incomplete in X_Keys.txt.' }
        'CONFIG_ERROR'         { return 'Configuration problem. Check X_Config.txt and Python packages.' }
        'MEDIA_FILE_NOT_FOUND' { return 'A selected file no longer exists. Re-select it.' }
        'MEDIA_UPLOAD_FAILED'  { return 'A media upload failed. Nothing was posted. Post again.' }
        'POST_CREATE_FAILED'   { return 'X refused the post. Nothing was posted.' }
        'POST_RESULT_UNKNOWN'  { return 'The post may or may not exist on X. CHECK X BEFORE POSTING AGAIN.' }
        default                { return 'See the debug log for details.' }
    }
}
```

```
function HandlePostingError {
    param($Result)
    $s = $script:AppState
    $code = [string]$Result.Code
    if ($Result.OutcomeUnknown) { $s.UnknownOutcomePending = $true }
    WriteHistoryRow -RequestId $s.RequestId -Status 'Failed' -Detail $code
    WriteDebugOutput "[RequestId=$($s.RequestId)] PS1 ERROR Code=$code Source=$($Result.Source)"
    UpdateGui -GuiState 'Error'
    $text = "Posting failed ($code).`r`n$($Result.Message)`r`n`r`n$(Get-ErrorAdvice $code)"
    if ($Result.OutcomeUnknown) {
        $text += "`r`n`r`nThe post may already exist. Verify on X before posting again."
    }
    ShowMessage -Text $text -Title 'XPoster' -Icon 'Error'
    UpdateGui -GuiState 'Idle'               # inputs are kept so the user can retry manually
}

# ============================ GuiController ============================
function DetermineStartupMode {
    param($Config)
    $needed = @($Config.PythonPath, $Config.PythonScriptPath)
    if ($script:ExtraPythonArgs -notmatch '--fake') { $needed += $Config.KeysFilePath }  # mock needs no keys
    foreach ($p in $needed) {
        if (-not (Test-Path -LiteralPath $p -PathType Leaf)) { return "ConfigProblem: missing $p" }
    }
    return 'Ready'
}

function InitializeGui {
    param([string]$StartupMode)
    $f = New-Object System.Windows.Forms.Form
    $f.Text = 'XPoster'; $f.Size = New-Object System.Drawing.Size(560, 520)
    $f.StartPosition = 'CenterScreen'; $f.MinimumSize = $f.Size
    $script:Form = $f

    $t = New-Object System.Windows.Forms.TextBox
    $t.Multiline = $true; $t.AcceptsReturn = $true; $t.ScrollBars = 'Vertical'
    $t.Location = '12,12'; $t.Size = '520,150'; $t.Anchor = 'Top,Left,Right'
    $script:txtPost = $t

    $l = New-Object System.Windows.Forms.ListBox
    $l.Location = '12,174'; $l.Size = '520,110'; $l.Anchor = 'Top,Left,Right'
    $script:lstMedia = $l
```

```
    $script:btnAttach = New-Object System.Windows.Forms.Button
    $script:btnAttach.Text = 'Attach...'; $script:btnAttach.Location = '12,292'
    $script:btnRemove = New-Object System.Windows.Forms.Button
    $script:btnRemove.Text = 'Remove'; $script:btnRemove.Location = '100,292'
    $script:btnPost = New-Object System.Windows.Forms.Button
    $script:btnPost.Text = 'Post'; $script:btnPost.Location = '440,292'
    $script:btnPost.Anchor = 'Top,Right'

    $script:lblFile = New-Object System.Windows.Forms.Label
    $script:lblFile.Location = '12,340'; $script:lblFile.Size = '520,20'
    $script:prgFile = New-Object System.Windows.Forms.ProgressBar
    $script:prgFile.Location = '12,364'; $script:prgFile.Size = '520,22'
    $script:prgFile.Minimum = 0; $script:prgFile.Maximum = 100
    $script:lblStatus = New-Object System.Windows.Forms.Label
    $script:lblStatus.Location = '12,396'; $script:lblStatus.Size = '520,40'

    $f.Controls.AddRange(@($t, $l, $script:btnAttach, $script:btnRemove, $script:btnPost,
        $script:lblFile, $script:prgFile, $script:lblStatus))

    $t.Add_TextChanged({ UpdatePostButtonState })
    $script:btnAttach.Add_Click({ AttachMedia })
    $script:btnRemove.Add_Click({ RemoveMedia })
    $script:btnPost.Add_Click({ StartPost })
    $f.Add_FormClosing({
        param($sender, $e)
        if ($script:AppState.PostInProgress) {
            $e.Cancel = $true
            ShowMessage -Text 'A post is running. Wait until it finishes.' -Icon 'Warning'
        }
    })
    $script:PumpTimer = New-Object System.Windows.Forms.Timer
    $script:PumpTimer.Interval = $script:PumpIntervalMs
    $script:PumpTimer.Add_Tick({ PumpRunner })

    UpdateGui -GuiState 'Idle'
    if ($StartupMode -ne 'Ready') {
        $script:lblStatus.Text = $StartupMode
        $script:btnPost.Enabled = $false
        $script:StartupBlocked = $true
    }
}

function AttachMedia {
    $dlg = New-Object System.Windows.Forms.OpenFileDialog
    $dlg.Multiselect = $true
    $dlg.Filter = 'Media|*.jpg;*.jpeg;*.png;*.webp;*.gif;*.mp4;*.mov'
    if ($dlg.ShowDialog() -ne 'OK') { return }
    foreach ($path in $dlg.FileNames) {
        $kind = Get-MediaKind $path
        $existing = @($script:SelectedMedia | ForEach-Object { Get-MediaKind $_ })
        if ($script:SelectedMedia.Contains($path)) { continue }
        if ($null -eq $kind) { ShowMessage -Text "Unsupported file type: $path" -Icon 'Warning'; continue }
        if ($script:SelectedMedia.Count -ge $script:MaxMedia) {
            ShowMessage -Text "At most $($script:MaxMedia) media files." -Icon 'Warning'; break }
        if (($kind -ne 'image' -and $existing.Count -gt 0) -or ($existing -contains 'gif') -or
            ($existing -contains 'video')) {
            ShowMessage -Text 'A GIF or video must be the only media item.' -Icon 'Warning'; continue }
        $script:SelectedMedia.Add($path)
        [void]$script:lstMedia.Items.Add([System.IO.Path]::GetFileName($path))
    }
    UpdatePostButtonState
}

function RemoveMedia {
    $i = $script:lstMedia.SelectedIndex
    if ($i -lt 0) { return }
    $script:SelectedMedia.RemoveAt($i)
    $script:lstMedia.Items.RemoveAt($i)
    UpdatePostButtonState
}

function UpdatePostButtonState {
    $hasContent = (-not [string]::IsNullOrWhiteSpace($script:txtPost.Text)) -or
                  ($script:SelectedMedia.Count -gt 0)
    $blocked = $script:AppState.PostInProgress -or $script:StartupBlocked
    $script:btnPost.Enabled = $hasContent -and -not $blocked
}
```

```
function UpdateGui {
    param([string]$GuiState)
    $busy = $script:AppState.PostInProgress
    $script:txtPost.ReadOnly = $busy
    $script:btnAttach.Enabled = -not $busy
    $script:btnRemove.Enabled = -not $busy
    switch ($GuiState) {
        'Idle'         { $script:lblStatus.Text = 'Ready.'; $script:lblFile.Text = ''
                         $script:prgFile.Value = 0 }
        'Starting'     { $script:lblStatus.Text = 'Uploader started.' }
        'TextComplete' { $script:lblStatus.Text = 'Text accepted.' }
        'Posting'      { $script:lblFile.Text = 'Publishing post...'; $script:prgFile.Value = 100
                         $script:lblStatus.Text = 'Creating the post on X.' }
        'Finishing'    { $script:lblStatus.Text = 'Finishing...' }
        'Success'      { $script:lblStatus.Text = 'Published.' }
        'Error'        { $script:lblStatus.Text = 'Posting failed.' }
    }
    UpdatePostButtonState
}

function UpdateUploadProgress {
    param([int]$CurrentFileIndex, [int]$TotalFiles, [int]$CurrentFileProgress,
          [string]$CurrentFileName)
    $script:lblFile.Text = "Uploading file $CurrentFileIndex of ${TotalFiles}: $CurrentFileName " +
        "($CurrentFileProgress%)"
    $script:prgFile.Value = [Math]::Max(0, [Math]::Min(100, $CurrentFileProgress))
}

# ============================ StartPost ============================
function StartPost {
    $s = $script:AppState
    if ($s.PostInProgress) { return }
    $text = $script:txtPost.Text
    $selected = @($script:SelectedMedia)
    if ([string]::IsNullOrWhiteSpace($text) -and $selected.Count -eq 0) { return }
    if ($s.UnknownOutcomePending) {
        $answer = [System.Windows.Forms.MessageBox]::Show($script:Form,
            'The previous post may already exist on X. Have you checked, and still want to post?',
            'XPoster', 'YesNo', 'Warning')
        if ($answer -ne 'Yes') { return }
        $s.UnknownOutcomePending = $false
    }
    $s.PostInProgress = $true
    UpdateGui -GuiState 'Starting'
    try {
        $requestId = (Get-Date).ToString('yyyyMMdd_HHmmss_fff')
        $record = ExportPostRecord -RequestId $requestId -PostText $text -SelectedMedia $selected
        $request = New-PythonPostRequest -RequestId $requestId -PostText $text -ExportedMedia $record.Media
        $problem = Test-PythonPostRequest $request
        if ($problem) { throw $problem }
    }
    catch {
        $s.PostInProgress = $false
        WriteDebugOutput "[PS] StartPost preparation failed: $($_.Exception.Message)"
        ShowMessage -Text "Cannot start the post: $($_.Exception.Message)" -Icon 'Error'
        UpdateGui -GuiState 'Idle'
        return
    }
    Reset-OperationState -Request $request
    WriteDebugOutput ("[RequestId={0}] PS1 start TextLength={1} TotalFiles={2}" -f
        $requestId, $text.Length, $request.TotalFiles)
    RunPythonEngine -Config $script:Config -Request $request -AppState $s
}

# ============================ MainController ============================
try {
    $script:Config = ReadConfig -Path $script:ConfigPath
    $logScript = $script:Config.DebugLogFunctionPath
    . $logScript                                  # dot-source at SCRIPT scope (not inside a function)
    WriteDebugOutput "XPoster started. DebugLogMode=$($script:Config.DebugLogMode)"
    $mode = DetermineStartupMode -Config $script:Config
    InitializeGui -StartupMode $mode
    [void]$script:Form.ShowDialog()
}
catch {
    [void][System.Windows.Forms.MessageBox]::Show($_.Exception.Message, 'XPoster startup', 'OK', 'Error')
    exit 1
}
```

Appendix C. xposter_conformance.py (complete)

Reference protocol client. It implements the same state machine and timeouts as the PowerShell router and runs tests A to O against `XPoster.py --fake`.

```
#!/usr/bin/env python3
"""Reference PS1-side client for XPoster protocol v1.

Mirrors the PowerShell state machine so the Python engine can be tested
on any machine without PowerShell. Usage: python xposter_conformance.py
"""
import json
import os
import subprocess
import sys
import tempfile
import threading
import queue
import time

HERE = os.path.dirname(os.path.abspath(__file__))
ENGINE = os.path.join(HERE, "XPoster.py")
IDLE_TIMEOUT = 3.0   # seconds; the PowerShell default is 90


def make_request(request_id, text, files):
    media = [{"MediaId": i + 1, "FileIndex": i + 1, "FileName": os.path.basename(p),
              "FilePath": p} for i, p in enumerate(files)]
    return {"ProtocolVersion": 1, "RequestId": request_id, "Text": text,
            "TotalFiles": len(media), "Media": media}


class Outcome(object):
    def __init__(self):
        self.events, self.code, self.terminal = [], None, None
        self.stderr, self.stdout_raw, self.exit_code = [], [], None
        self.media_ids, self.post_id = {}, None
        self.ignored_after_terminal = 0


def run_case(request, extra_args=(), raw_stdin=None):
    out = Outcome()
    cmd = [sys.executable, "-u", ENGINE, "--fake"] + list(extra_args)
    proc = subprocess.Popen(cmd, stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE)
    q = queue.Queue()

    def pump(stream, kind):
        for line in iter(stream.readline, b""):
            q.put((kind, line))
        q.put((kind + "_EOF", None))

    for stream, kind in ((proc.stdout, "OUT"), (proc.stderr, "ERR")):
        threading.Thread(target=pump, args=(stream, kind), daemon=True).start()
    if raw_stdin is None:
        raw_stdin = (json.dumps(request, separators=(",", ":")) + "\n").encode("utf-8")
    data = raw_stdin
    proc.stdin.write(data)
    proc.stdin.flush()
    proc.stdin.close()

    rid = request["RequestId"] if request else None
    phase, nxt, eofs = "Starting", 0, 0
    total = request["TotalFiles"] if request and "TotalFiles" in request else 0
    last = time.time()

    def fail(code):
        out.terminal, out.code = "SYNTHETIC", code
        proc.kill()
```

```
    while eofs < 2:
        try:
            kind, line = q.get(timeout=0.2)
        except queue.Empty:
            if out.terminal is None and time.time() - last > IDLE_TIMEOUT:
                fail("OPERATION_TIMEOUT")
                break
            continue
        if kind == "ERR":
            out.stderr.append(line.decode("utf-8", "replace").rstrip())
            continue
        if kind.endswith("_EOF"):
            eofs += 1
            continue
        out.stdout_raw.append(line)
        last = time.time()
        text = line.decode("utf-8").rstrip("\r\n")
        if out.terminal is not None:
            out.ignored_after_terminal += 1
            continue
        if text.strip() == "":
            continue
        try:
            msg = json.loads(text)
            ev = msg["Event"]
        except Exception:
            fail("PROTOCOL_INVALID_STDOUT_JSON")
            continue
        # An ERROR may carry RequestId=null when the request itself was unreadable.
        null_ok = ev == "ERROR" and msg.get("RequestId") is None
        if msg.get("RequestId") != rid and not null_ok:
            fail("PROTOCOL_REQUEST_ID_MISMATCH")
            continue
        out.events.append(ev)
        if ev == "READY" and phase == "Starting":
            phase = "Ready"
        elif ev == "TEXT_COMPLETE" and phase == "Ready":
            phase = "Media"
        elif ev == "PROGRESS" and phase == "Media" and nxt < total and msg["MediaId"] == nxt + 1:
            pass
        elif ev == "MEDIA_COMPLETE" and phase == "Media" and nxt < total and msg["MediaId"] == nxt + 1:
            out.media_ids[msg["MediaId"]] = msg["XMediaId"]
            nxt += 1
        elif ev == "POSTING" and phase == "Media" and nxt == total and msg["MediaCount"] == total:
```

```
            phase = "Posting"
        elif ev == "SUCCESS" and phase == "Posting":
            out.terminal, out.post_id = "SUCCESS", msg["PostId"]
        elif ev == "ERROR":
            out.terminal, out.code = "ERROR", msg["Code"]
            out.error = msg
        else:
            fail("PROTOCOL_SEQUENCE_ERROR" if ev in (
                "READY", "TEXT_COMPLETE", "PROGRESS", "MEDIA_COMPLETE", "POSTING", "SUCCESS")
                else "PROTOCOL_UNEXPECTED_EVENT")
    try:
        out.exit_code = proc.wait(timeout=5)
    except subprocess.TimeoutExpired:
        proc.kill()
        out.exit_code = proc.wait()
    if out.terminal is None:
        out.terminal, out.code = "SYNTHETIC", "PYTHON_EXITED_UNEXPECTEDLY"
    out.phase_at_end = phase
    return out


def main():
    tmp = tempfile.mkdtemp()
    files = []
    for i in range(1, 5):
        path = os.path.join(tmp, "image%d.jpg" % i)
        with open(path, "wb") as handle:
            handle.write(b"\xff\xd8\xff" + b"0" * 100)
        files.append(path)
    gif = os.path.join(tmp, "anim.gif")
    with open(gif, "wb") as handle:
        handle.write(b"GIF89a" + b"0" * 50)
    results = []

    def check(name, condition, detail=""):
        results.append(condition)
        print("%-4s %s %s" % ("PASS" if condition else "FAIL", name, detail))
```

```
    seq = lambda o: o.events
    o = run_case(make_request("T_A", "text only", []))
    check("A text only", o.terminal == "SUCCESS"
          and seq(o) == ["READY", "TEXT_COMPLETE", "POSTING", "SUCCESS"])
    o = run_case(make_request("T_B", "", files[:1]))
    check("B one media, no text", o.terminal == "SUCCESS" and o.events.count("PROGRESS") == 6
          and o.events[-3:] == ["MEDIA_COMPLETE", "POSTING", "SUCCESS"]
          and o.media_ids == {1: "101000000000000000"})
    o = run_case(make_request("T_C", "four", files))
    check("C four media", o.terminal == "SUCCESS" and sorted(o.media_ids) == [1, 2, 3, 4]
          and o.events.count("MEDIA_COMPLETE") == 4)
    o = run_case(make_request("T_D", "x", files[:3]), ["--fault", "upload_fail_2"])
    check("D error on media 2", o.terminal == "ERROR" and o.code == "MEDIA_UPLOAD_FAILED"
          and "POSTING" not in o.events and o.error.get("MediaId") == 2 and o.exit_code == 1)
    o = run_case(make_request("T_E", "x", files[:1]), ["--fault", "crash_mid"])
    check("E crash without ERROR", o.code == "PYTHON_EXITED_UNEXPECTEDLY" and o.exit_code == 3)
    o = run_case(make_request("T_F", "x", files[:1]), ["--fault", "garbage_stdout"])
    check("F invalid stdout", o.code == "PROTOCOL_INVALID_STDOUT_JSON")
    o = run_case(make_request("T_G", "x", files[:1]), ["--fault", "wrong_request_id"])
    check("G request id mismatch", o.code == "PROTOCOL_REQUEST_ID_MISMATCH")
    o = run_case(make_request("T_H", "   ", []))
    check("H empty post", o.code == "EMPTY_POST" and o.events == ["ERROR"] and o.exit_code == 1)
    text = u"Za\u017c\u00f3\u0142\u0107\r\n\u65e5\u672c\u8a9e \U0001F600\nline3"
    o = run_case(make_request("T_I", text, []))
    expect = len(text.replace("\r\n", "\n"))
    tc = [json.loads(l) for l in o.stdout_raw if b"TEXT_COMPLETE" in l][0]
    check("I unicode + multiline", o.terminal == "SUCCESS" and tc["CharacterCount"] == expect
          and all(max(l) < 128 for l in o.stdout_raw), "chars=%d" % expect)
    o = run_case(make_request("T_J", "x", files[:1]), ["--fault", "stderr_noise"])
    check("J stderr noise", o.terminal == "SUCCESS" and any("retry attempt" in s for s in o.stderr))
    o = run_case(make_request("T_K", "x", files[:1]), ["--fault", "hang"])
    check("K hang -> timeout", o.code == "OPERATION_TIMEOUT")
    o = run_case(make_request("T_L", "x", files[:2]), ["--fault", "skip_media_complete"])
    check("L sequence error", o.code == "PROTOCOL_SEQUENCE_ERROR")
    o = run_case(make_request("T_M", "x", []), ["--fault", "post_unknown"])
    check("M post result unknown", o.code == "POST_RESULT_UNKNOWN" and "POSTING" in o.events)
    o = run_case(make_request("T_N", "x", []), ["--fault", "extra_after_terminal"])
    check("N extra line after SUCCESS ignored", o.terminal == "SUCCESS" and o.ignored_after_terminal == 1)
    bad = make_request("T_O1", "x", [])
    bad["ProtocolVersion"] = 2
    o = run_case(bad)
    check("O1 unsupported version", o.code == "UNSUPPORTED_PROTOCOL_VERSION")
    o = run_case(make_request("T_O2", "x", []), raw_stdin=b"not json\n")
    check("O2 malformed request", o.code == "MALFORMED_REQUEST_JSON")

    miss = make_request("T_O3", "x", [os.path.join(tmp, "nope.jpg")])
    check("O3 file missing", run_case(miss).code == "MEDIA_FILE_NOT_FOUND")
    five = make_request("T_O4", "x", files + files[:1])
    check("O4 five media", run_case(five).code == "MEDIA_COUNT_INVALID")
    mix = make_request("T_O5", "x", [files[0], gif])
    check("O5 gif + image", run_case(mix).code == "MEDIA_COMBINATION_INVALID")
    long_text = make_request("T_O6", "y" * 281, [])
    check("O6 text too long", run_case(long_text).code == "TEXT_TOO_LONG")
    odd = make_request("T_O7", "x", [])
    odd["Media"] = None
    check("O7 invalid structure", run_case(odd).code == "INVALID_REQUEST")
    heb = os.path.join(tmp, u"\u05ea\u05de\u05d5\u05e0\u05d4.jpg")     # Hebrew file name
    with open(heb, "wb") as handle:
        handle.write(b"\xff\xd8\xff" + b"0" * 100)
    o = run_case(make_request("T_P", u"\u05e9\u05dc\u05d5\u05dd", [heb]))
    names = [json.loads(l)["FileName"] for l in o.stdout_raw if b"MEDIA_COMPLETE" in l]
    check("P Hebrew file name round trip", o.terminal == "SUCCESS" and names == [os.path.basename(heb)])
    print("\n%d/%d passed" % (sum(results), len(results)))
    return 0 if all(results) else 1


if __name__ == "__main__":
    sys.exit(main())
```